Data Structures and Algorithms

Lecture 7

Trees

Continue…

Dept. of ICT, Faculty of Technology- UoR, 2026

Example Application of Binary Tree

- Arithmetic Expression Tree
- Binary Search Tree
- Decision Tree
- AVL Tree
- Priority Queue
  - Binary Heap Tree

Binary tree basic operators

- **Pre-order Traversal** – Traverses a tree in a pre-order manner.
- **In-order Traversal** – Traverses a tree in an in-order manner.
- **Post-order Traversal** –Traverses a tree in a post-order manner.
- **Search** – Searches an element in a tree.
- **Insert** – Inserts an element in a tree.
- **Delete** – Delete an element in a tree.

Binary Tree Traversals

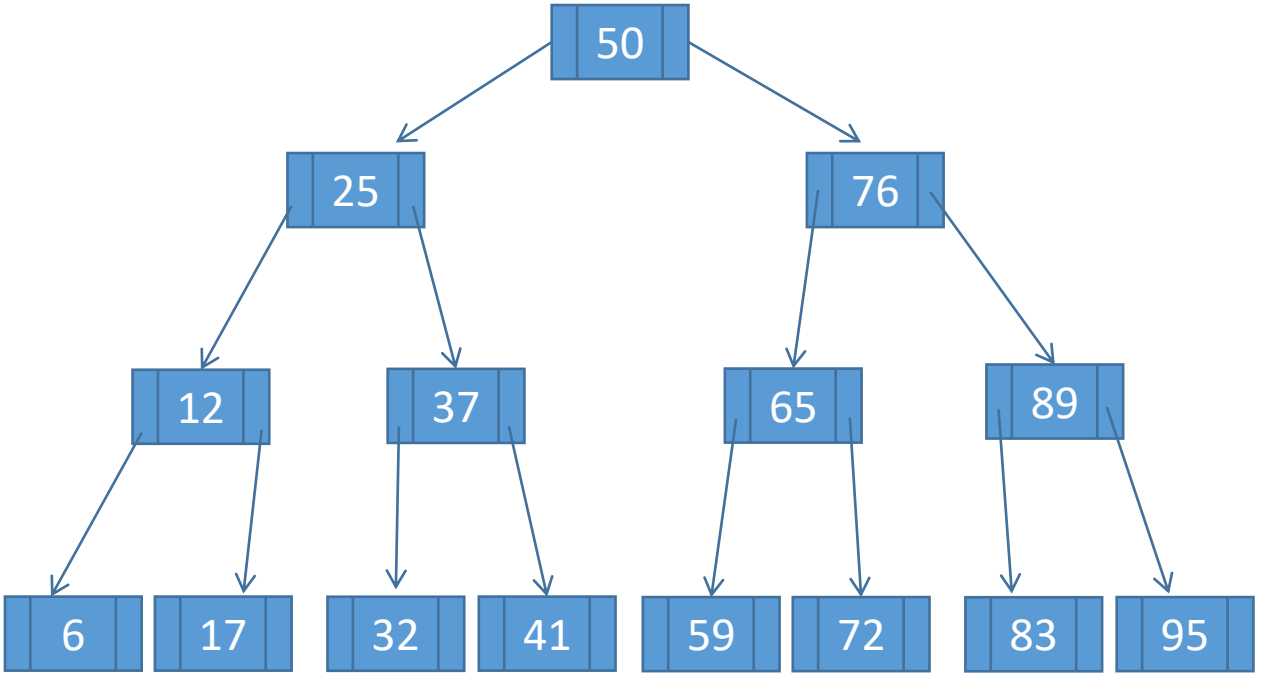
- Many algorithms require all nodes of a binary tree be visited and the contents of each node processed or examined.

- Three types of traversals
  - **preorder traversal:** process the root, then process all sub trees (left to right).
  - **inorder traversal:** process the left sub tree, process the root, process the right sub tree.
  - **postorder traversal:** process the left sub tree, process the right sub tree, then process the root.

Binary Tree Traversals

traversal:

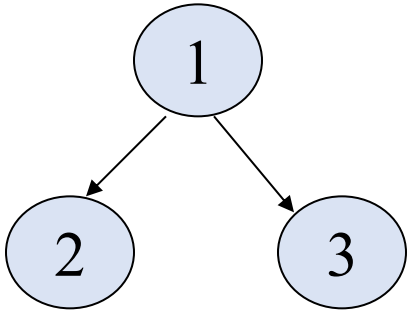
- Preorder
- Inorder
- Postorder



Binary Tree Traversals

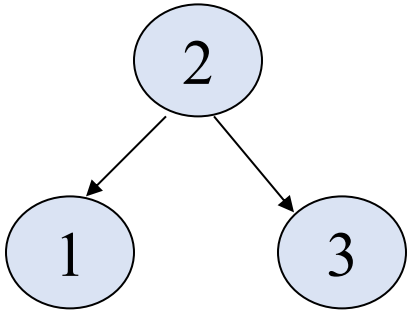
Preorder:
When you get to a node:
Print the node's data.
Then traverse its left subtree.
Then traverse its right subtree.

Node	1
Left	2
Right	3



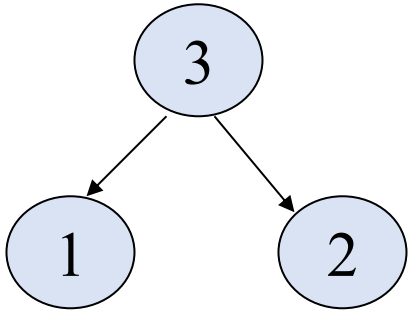
Inorder: Sorted order
When you get to a node: Traverse its left subtree.
Then print the node's data.
Then traverse its right subtree.

Left	1
Node	2
Right	3



Postorder:
When you get to a node: Traverse its left subtree.
Then traverse its right subtree.
Then print the node's data.

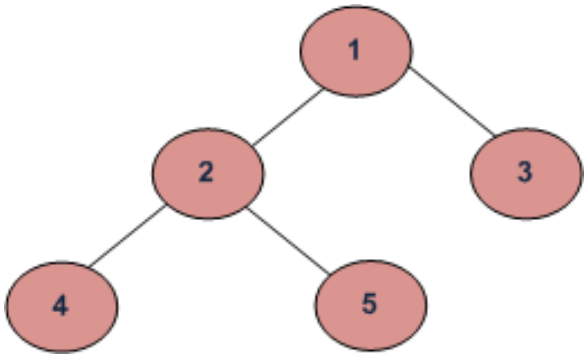
Left	1
Right	2
Node	3



Tree Traversals (Inorder, Preorder and Postorder)

Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways.

Following are the generally used ways for traversing trees.

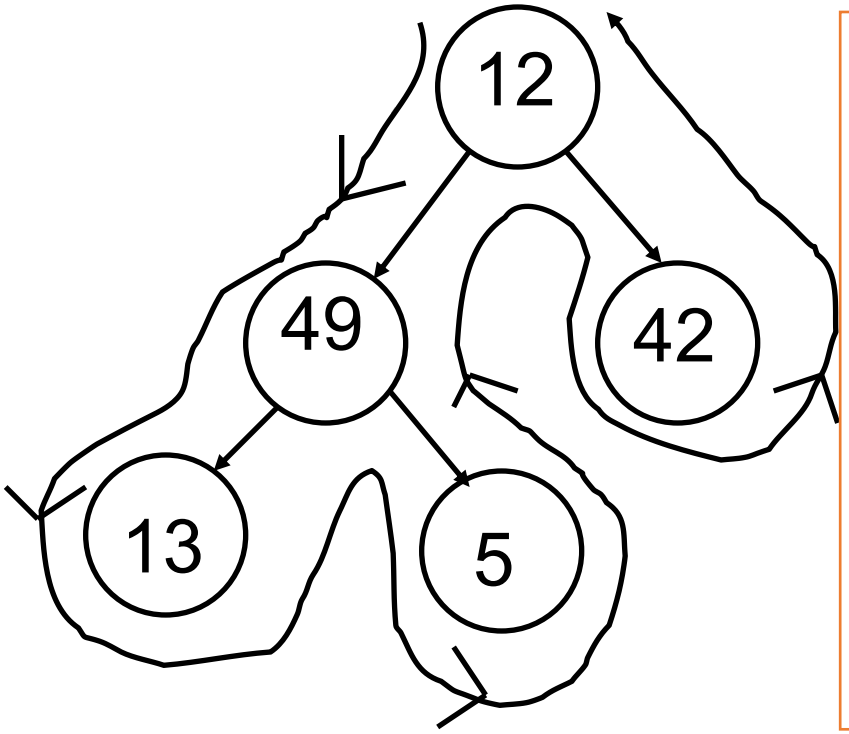


Depth First Traversals:

(a) Preorder (Root, Left, Right) : 1 2 4 5 3
(b) Inorder (Left, Root, Right) : 4 2 5 1 3
(c) Postorder (Left, Right, Root) : 4 5 2 3 1
Breadth First or Level Order Traversal : 1 2 3 4 5

Results of Traversals

- To determine the results of a traversal on a given tree draw a path around the tree.
  - start on the left side of the root and trace around the tree. The path should stay close to the tree.

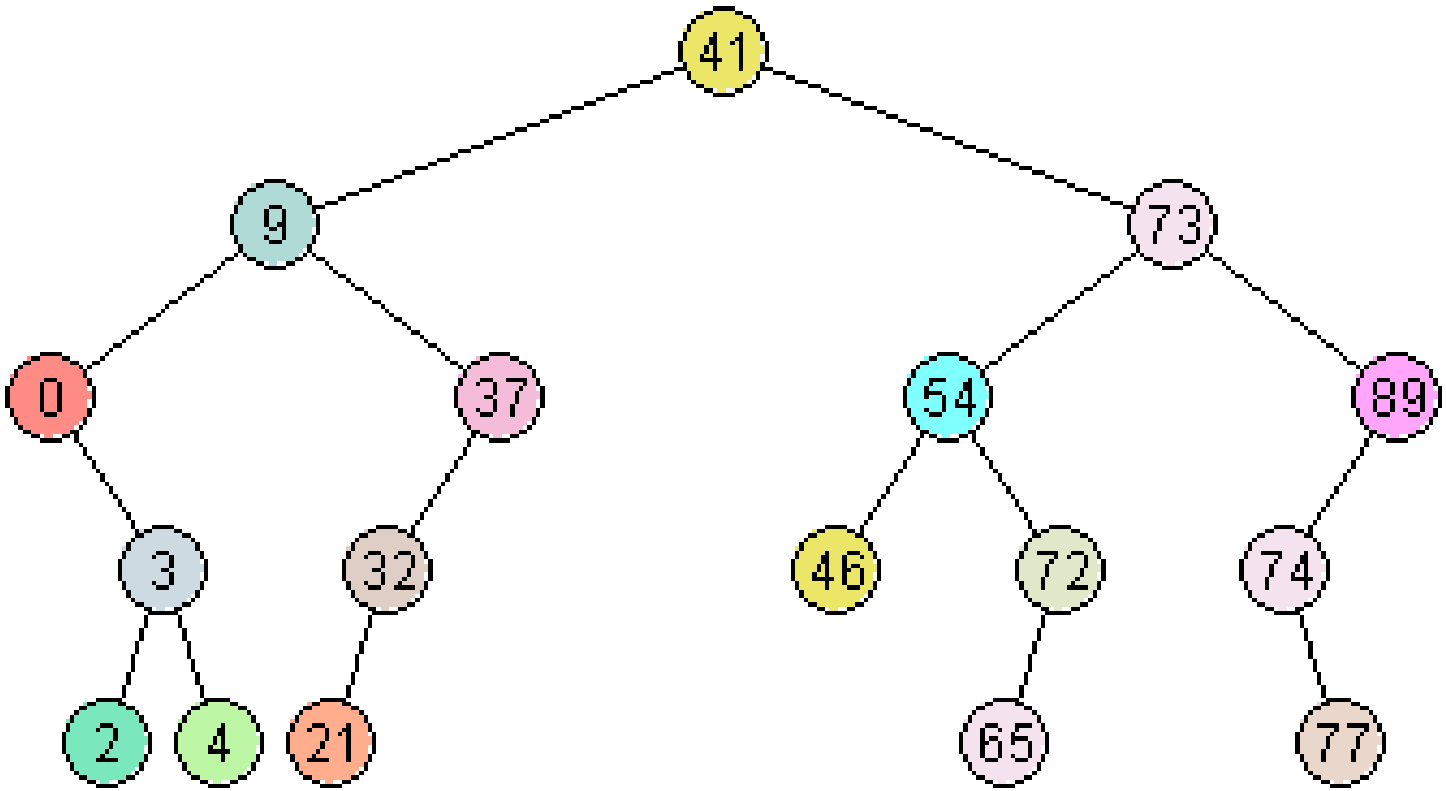


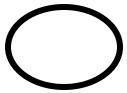
pre order: process when pass down left side of node
12 49 13 5 42

in order: process when pass underneath node
13 49 5 12 42

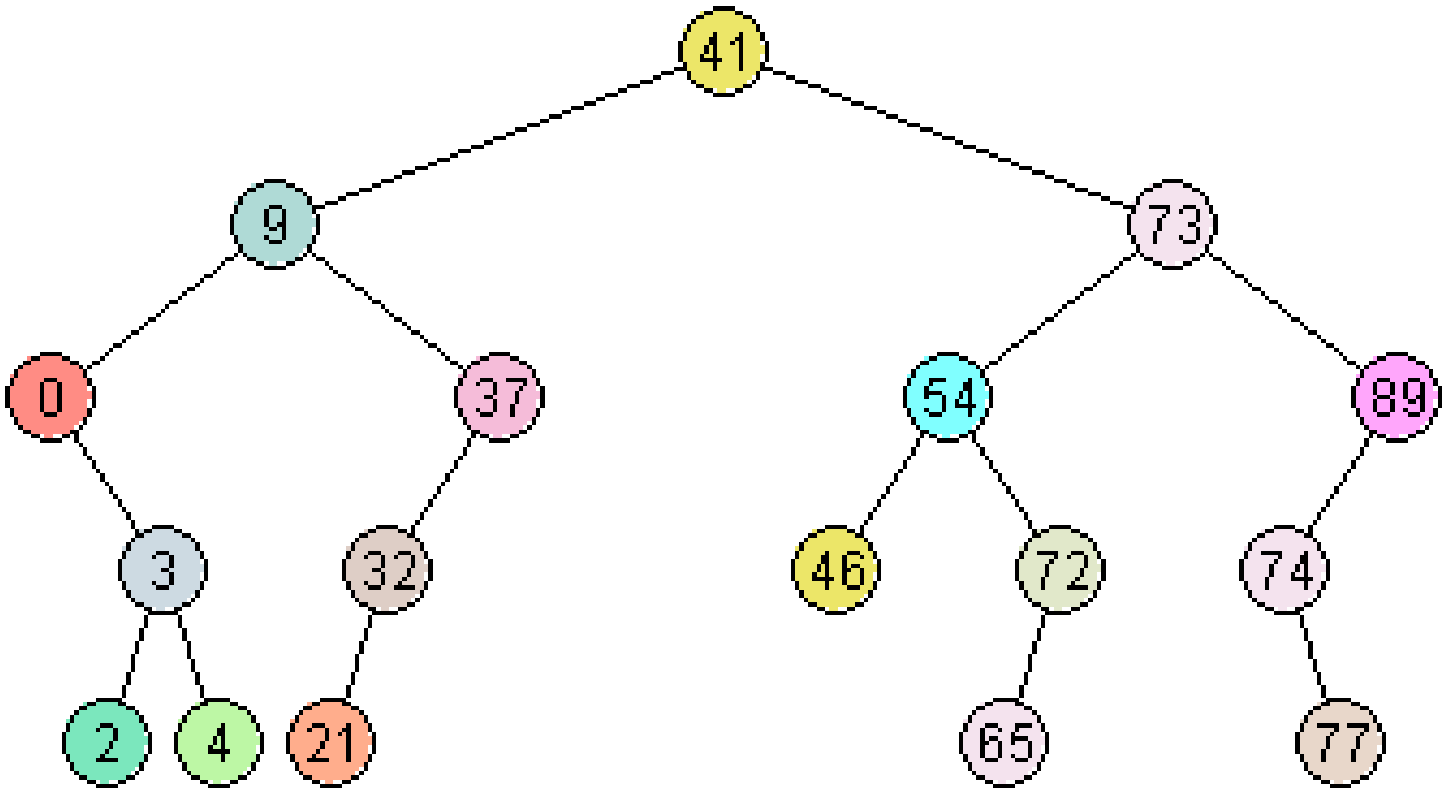
post order: process when pass up right side of node
13 5 49 42 12

Binary Search Tree – Pre Order



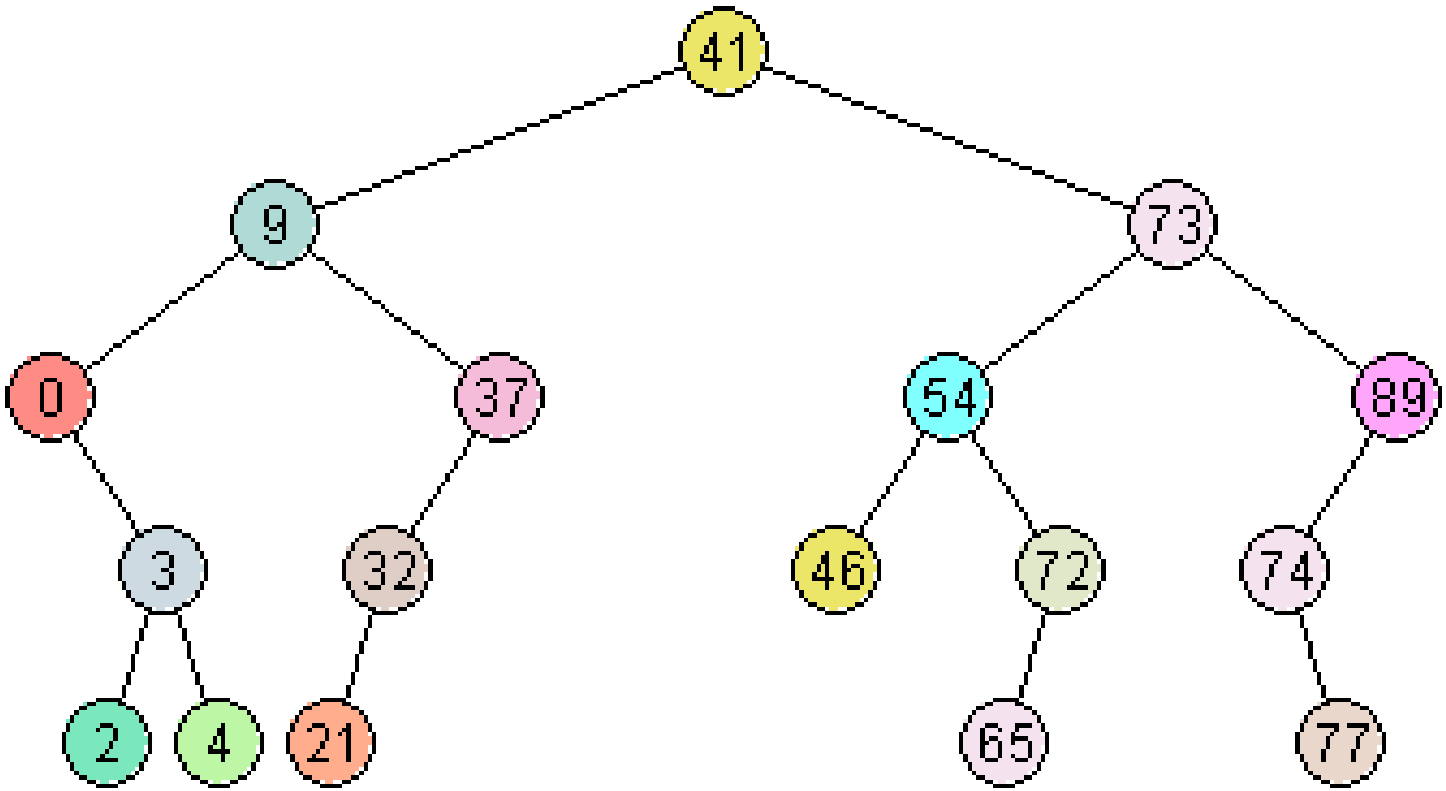


Binary Search Tree – Pre Order

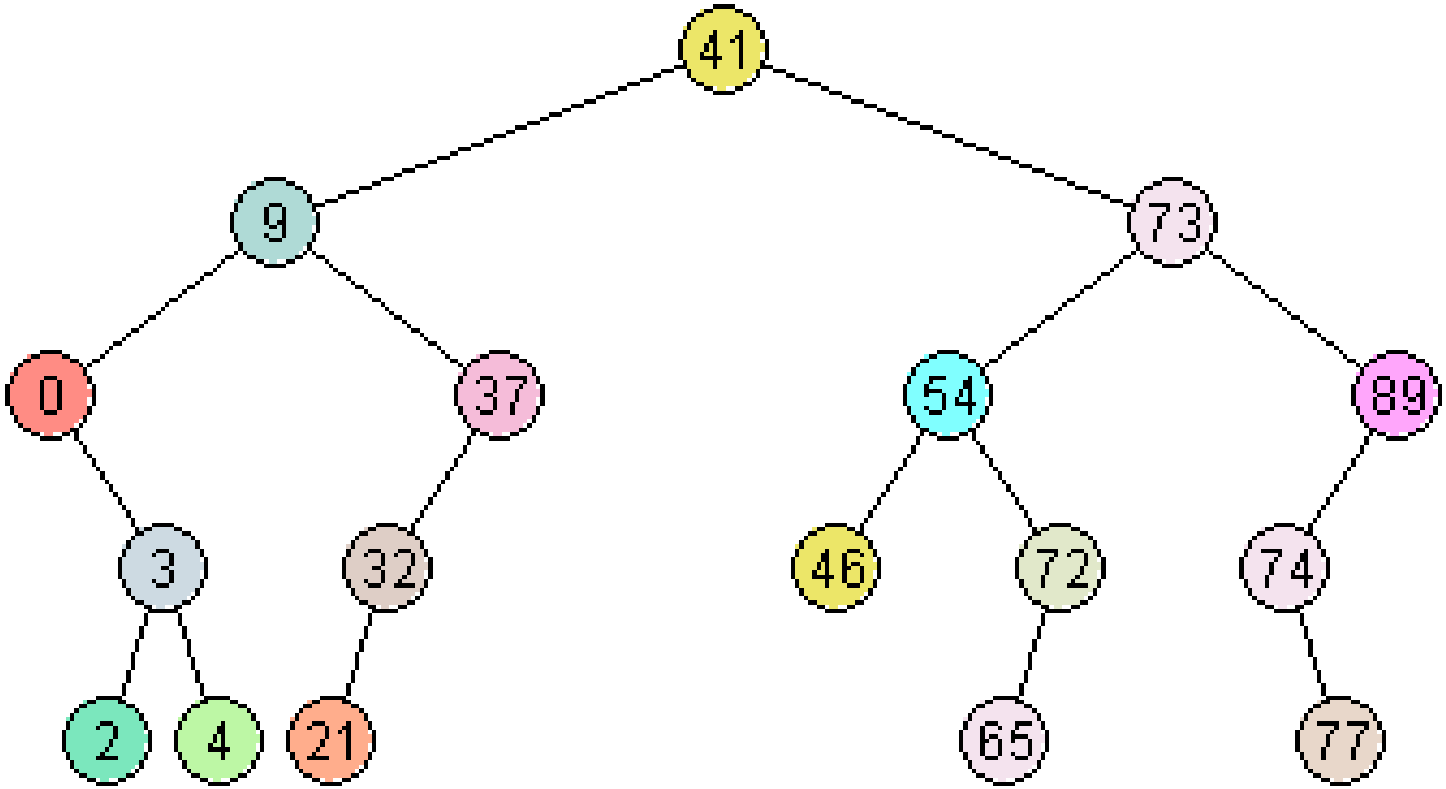


41 9 0 3 2 4 37 32 21 73 54 46 72 65 89 74 77

Binary Search Tree – Post Order



Binary Search Tree – Post Order



2 4 3 0 21 32 37 9 46 65 72 54 77 74 89 73 41

C programme

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node* left;
   struct node* right;
};

void inorder(struct node* root){
   if(root == NULL) return;
   inorder(root->left);
   printf("%d ->", root->data);
   inorder(root->right);
}

void preorder(struct node* root){
   if(root == NULL) return;
   printf("%d ->", root->data);
   preorder(root->left);
   preorder(root->right);
}
```

```
void postorder(struct node* root) {
   if(root == NULL) return;
   postorder(root->left);
   postorder(root->right);
   printf("%d ->", root->data);
}

struct node* createNode(value){
   struct node* newNode = malloc(sizeof(struct
node));
   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;

   return newNode;
}

struct node* insertLeft(struct node *root, int
value) {
   root->left = createNode(value);
   return root->left;
}
```

```
struct node* insertRight(struct node
*root, int value){
    root->right = createNode(value);
    return root->right;
}
int main(){
    struct node* root = createNode(1);
    insertLeft(root, 12);
    insertRight(root, 9);

    insertLeft(root->left, 5);
    insertRight(root->left, 6);

    printf("Inorder traversal \n");
    inorder(root);

    printf("\nPreorder traversal \n");
    preorder(root);

    printf("\nPostorder traversal \n");
    postorder(root);}
```

Infix, prefix and postfix notation

Consider the following mathematical expression:

$$4 \times (3 + 8)$$

This is written in what is called **infix notation**, in which the operators (+, - , x, /) are written *between* their operands. It is the usual notation that you are familiar with from mathematics.

Infix: $4 \times (3 + 8)$

Prefix: $\times 4 + 3\ 8$

Postfix: $4\ 3\ 8 + \times$

Examples

Infix: $4 \times (3 + 8)$

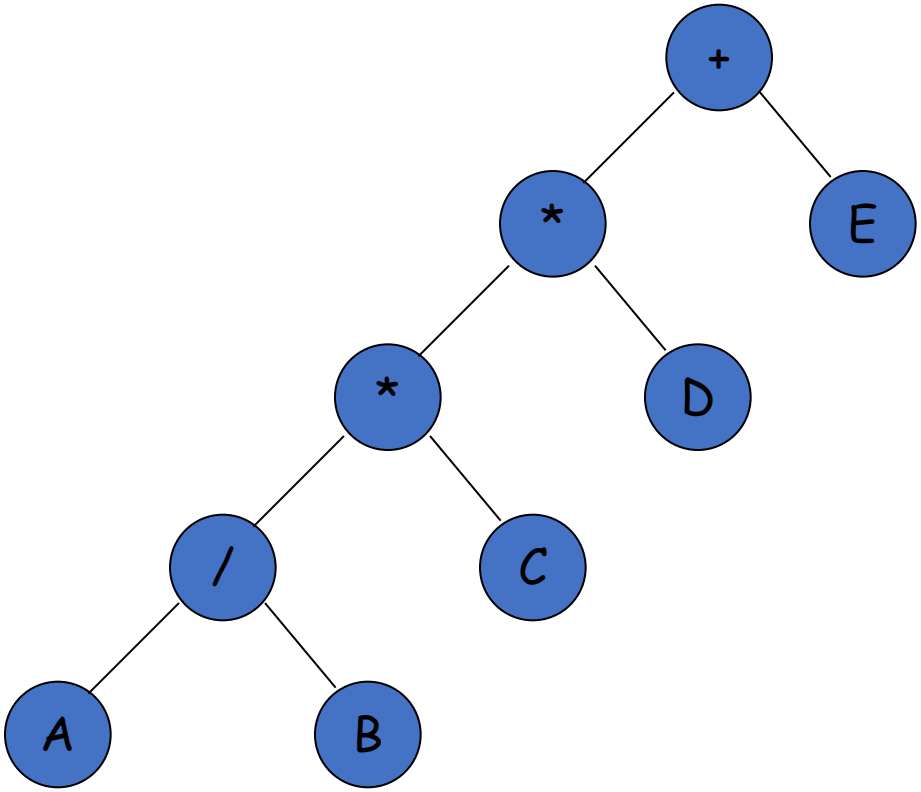
Prefix: $\times 4 + 3\ 8$

Postfix: $4\ 3\ 8 + \times$

Convert the following expressions to prefix and postfix:

1. $4 + 5$
2. $4 / 3 + 7$
3. $(5 + 2) \times (3 + 1)$
4. $4 / (3 + 6) + 5 \times 9$

Binary Tree With Arithmetic Expression



Binary Tree With Arithmetic Expression

Inorder Traversal: A/B*C*D+E

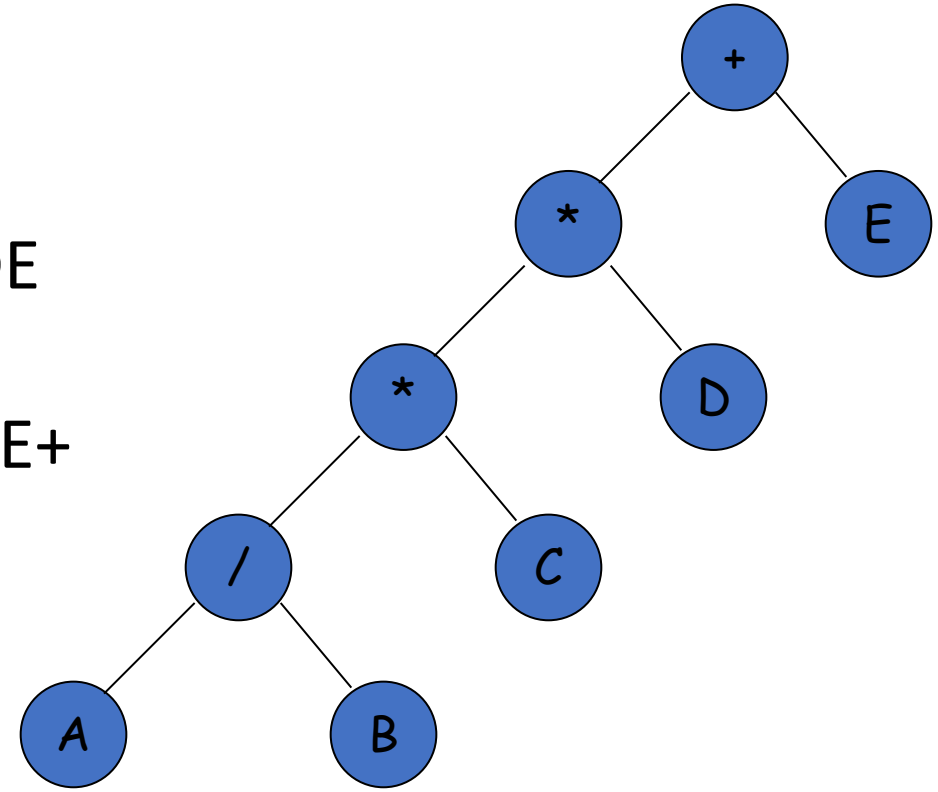
=> Infix form

Preorder Traversal: +**/ABCDE

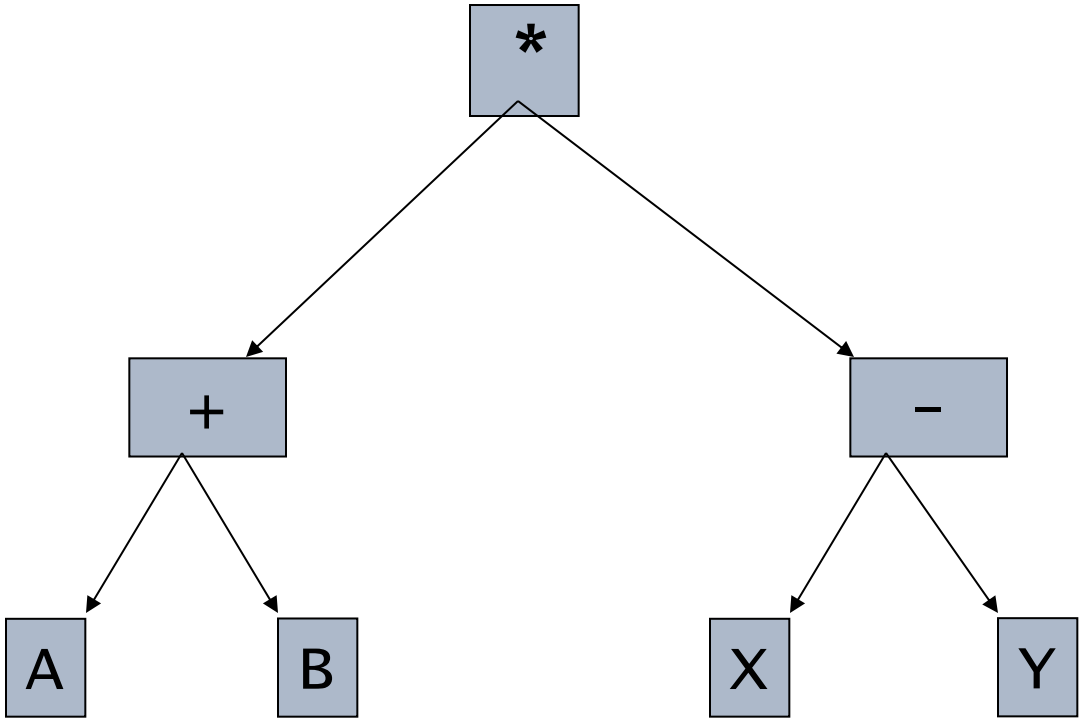
=> Prefix form

Postorder Traversal: AB/C*D*E+

=> Postfix form



An expression tree example...



UNDERSTAND TRAVERAL ORDER TO DISPLAY THE FOLLOWING

INFIX1: A+B*X-Y

INFIX2: ((A+B) * (X-Y))

PREFIX: *+AB-XY

POSTFIX: AB+XY- *

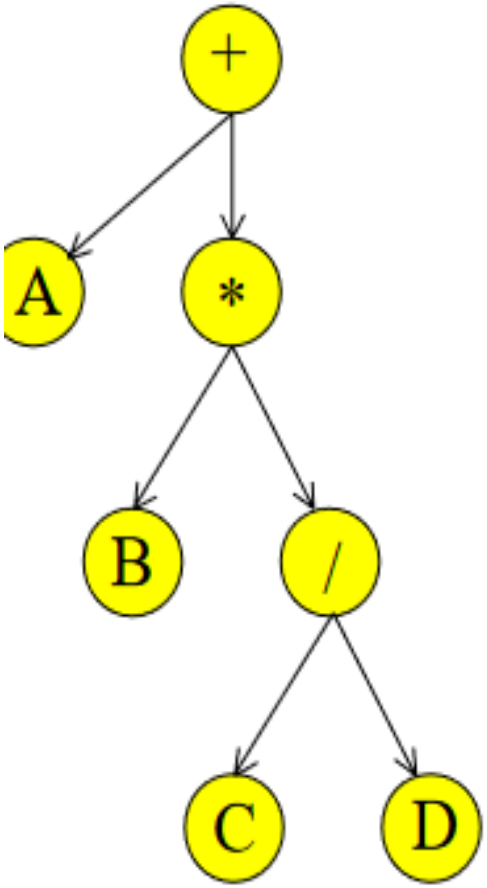
Exercise 1

Draw an expression tree for the following.

A + (B * (C / D))

Exercise 1: answer

A + (B * (C / D))



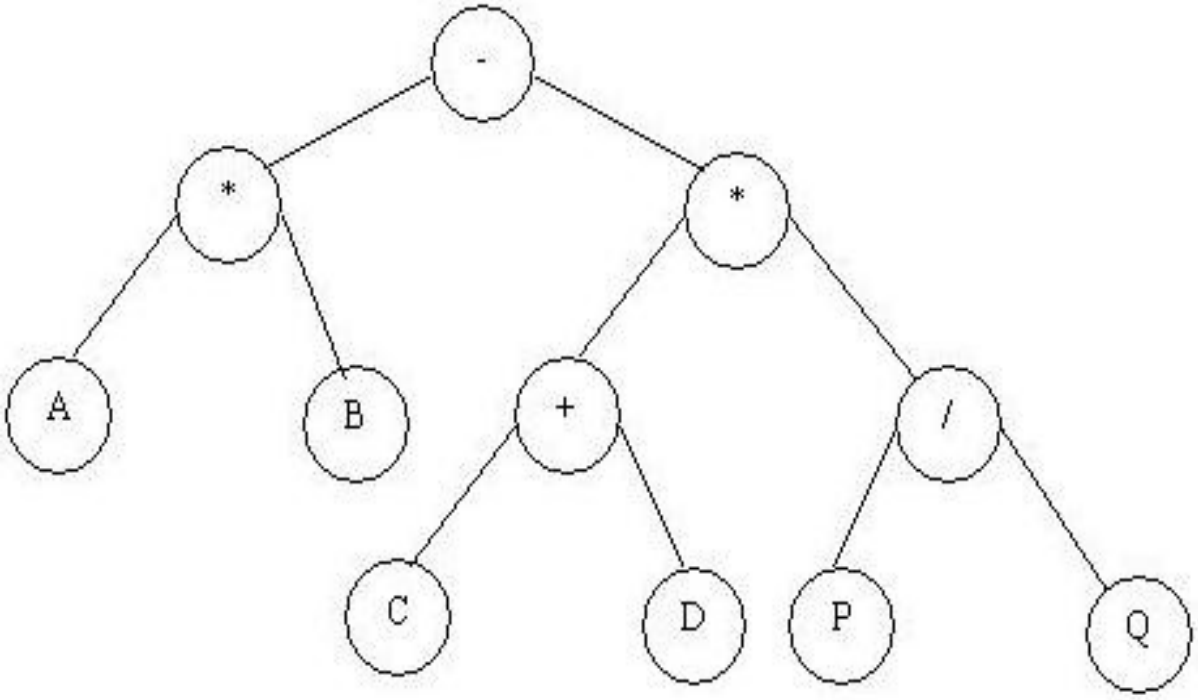
Exercise 2

Draw a binary Tree for the expression :

A * B – (C + D) * (P / Q)

Exercise 2: answer

A * B – (C + D) * (P / Q)



Home Exercises – create the trees

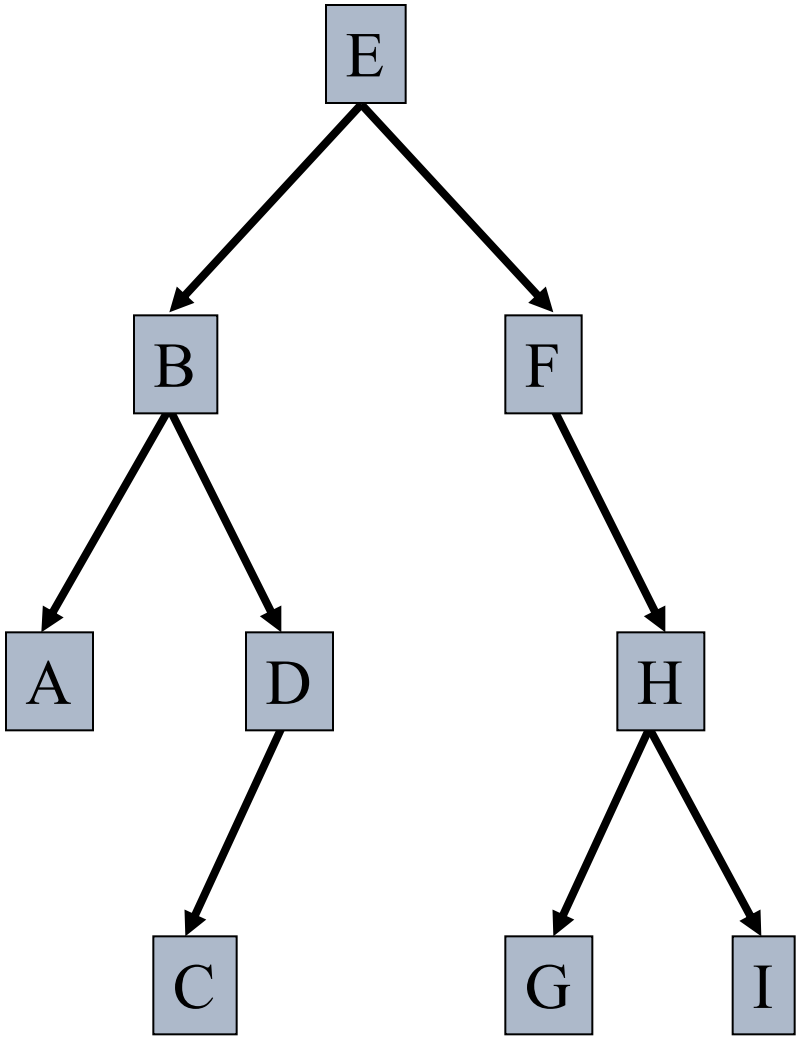
1. (a + b) / (c – d * e) + f + g * h / i
2. ((A + B) * (C + D)) / (E + F * H)

3. Represent the following expression in a binary tree.
[a + (b – c)] * [(d – e) / (f + g – h)]

4. Write the pre order and post order traversals of the resulting binary tree obtained

Height and Depth

- Node height is path length to the most distant descendant.
- Height of tree is height of root node.
- Depth is path length from the node to the root.



Height of E is 3

Height of B is 2

Height of D is 1

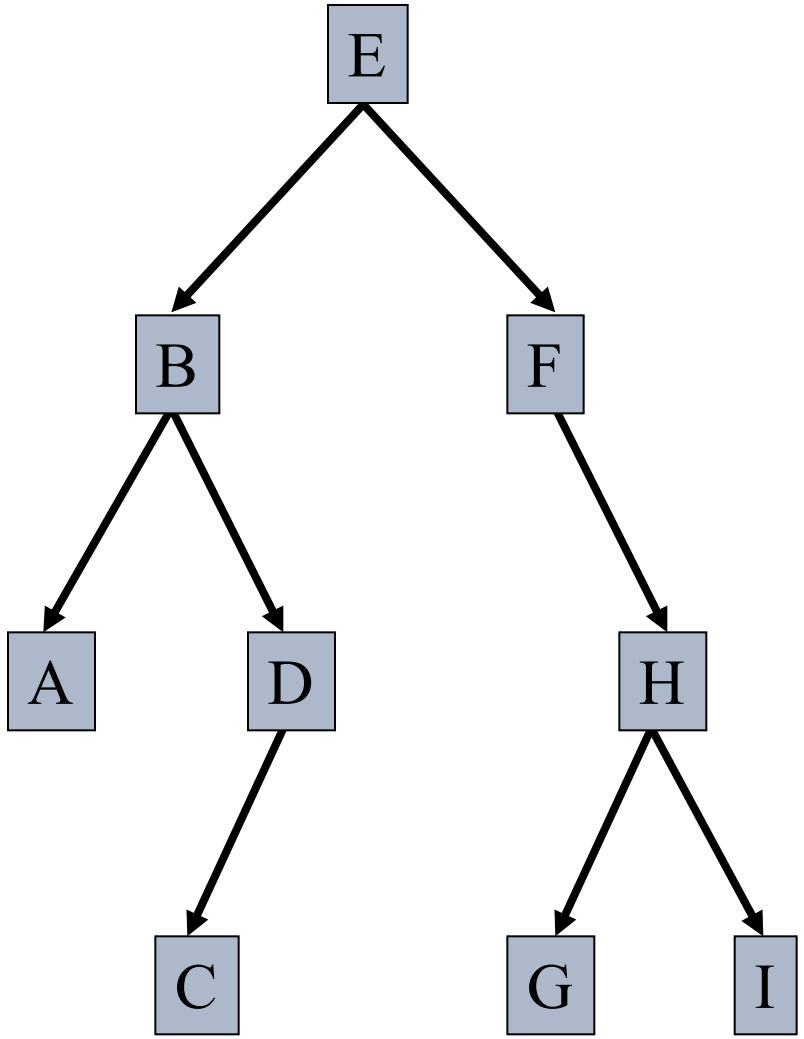
Depth of C is 3

Depth of F is 1

Levels

- Number of levels counts the number of nodes along any path

This tree has 4 levels, since there are 4 nodes along the longest path.



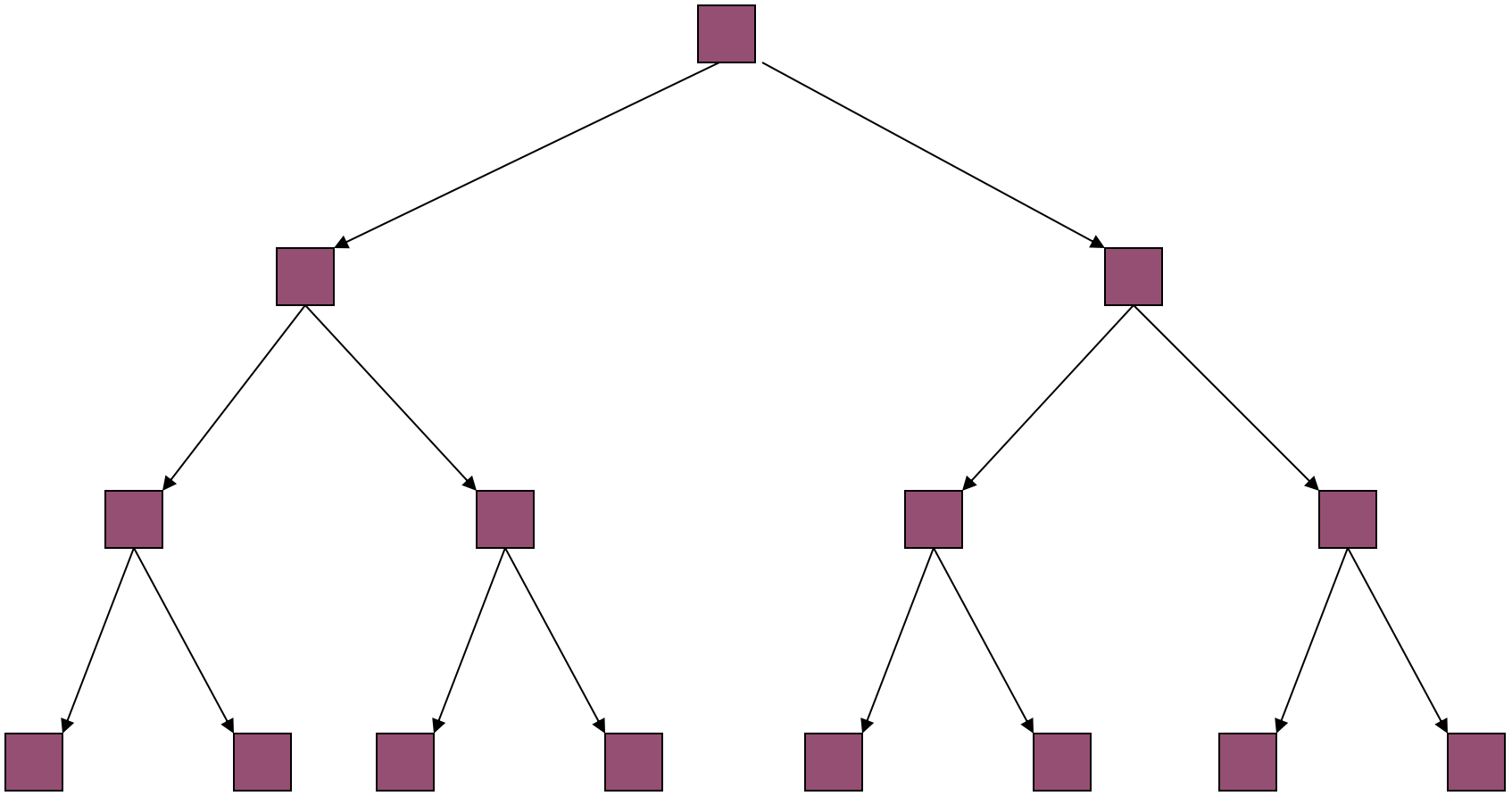
Binary Tree Complexity Analysis

- For a binary tree of height H and L=H+1 levels, there are $2^{L} - 1 = N$ nodes.
- When searching for a target, must visit at most L levels.
- The worst case search for completely balanced tree is $\log_2(N+1) = L$
- Time required is proportional to $\log_2 N$
- In contrast, time required for worst case linked list search is proportional to N

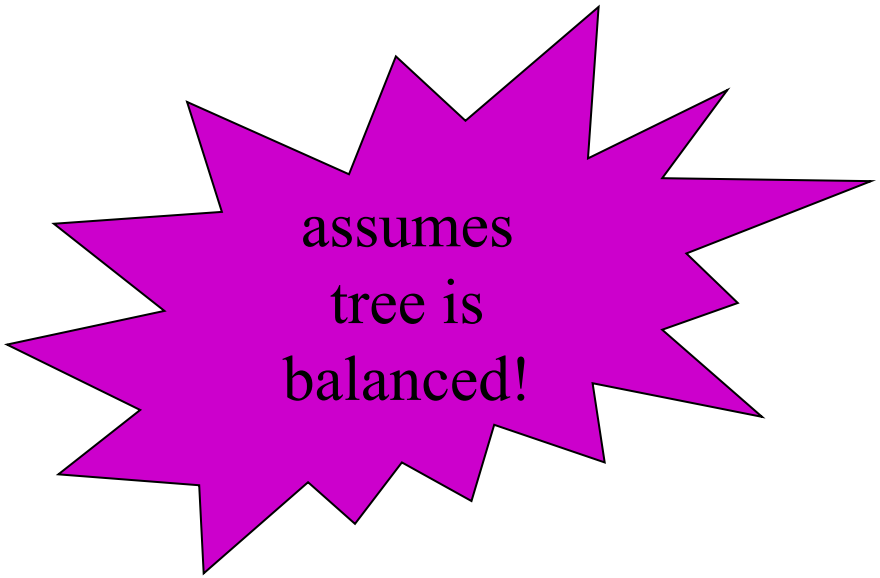
$2^4 - 1 = 15$ Nodes

15 Nodes in total
Height is 3 (links in max path)

4 Levels = (nodes in max path)
Levels is $\log_2(15+1)$



Linked list
versus
balanced
binary tree...



N	$\log_2 N$
2	1
4	2
8	3
16	4
32	5
64	6
128	7
256	8
512	9
1024	10
2048	11
4096	12
8192	14
16384	13
32768	15

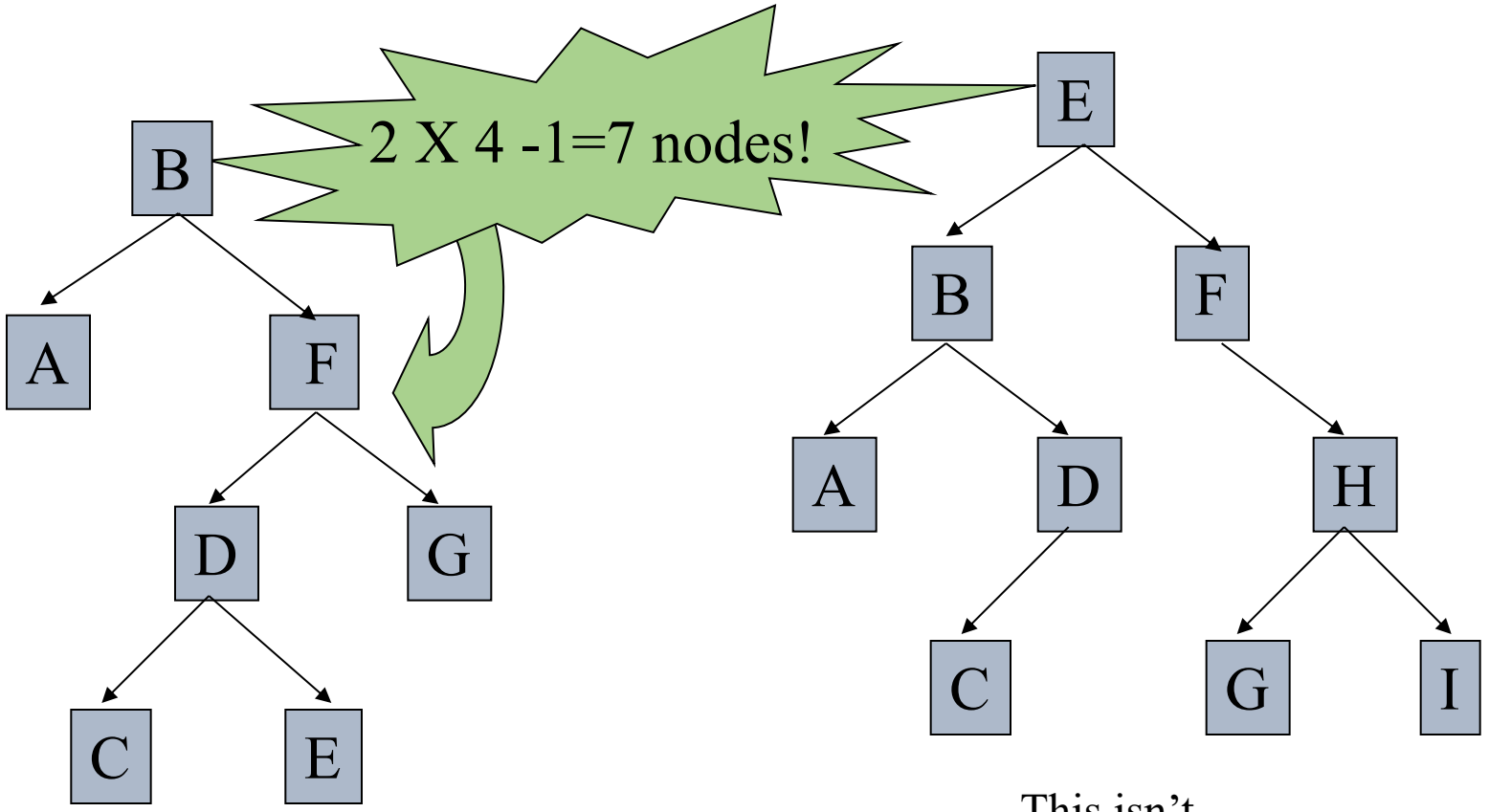
Types of binary trees

- Binary trees classifications:
  - strictly binary tree
  - complete binary tree
  - almost complete binary tree

- Classification is important when evaluating efficiency of processing

Strictly binary trees

- Every non-leaf node branches to two children.(A tree in which every node other than the leaves has two children)
- For n leaves there are always 2n-1 nodes.

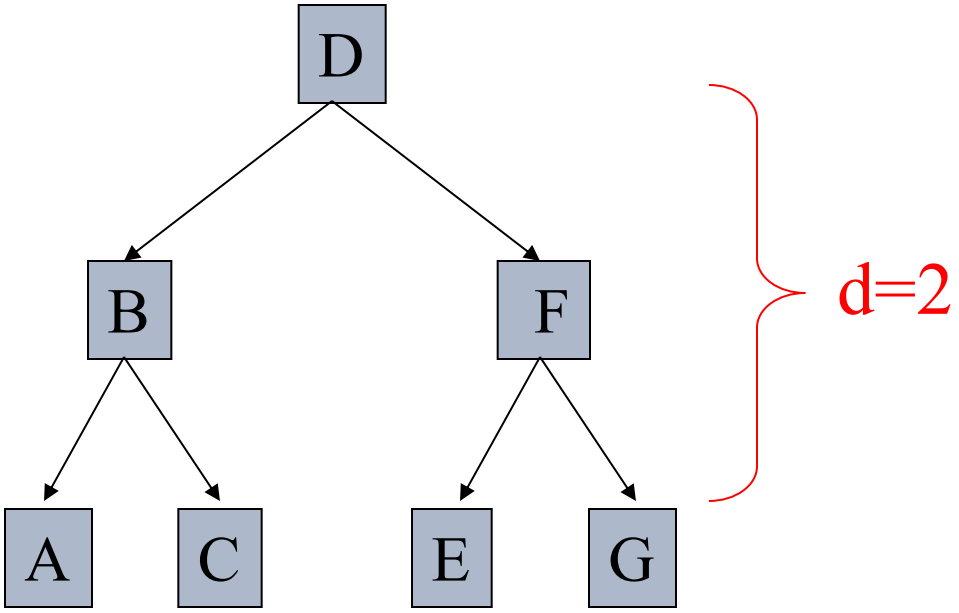


This is a strictly binary tree

This isn't

Complete Binary Trees

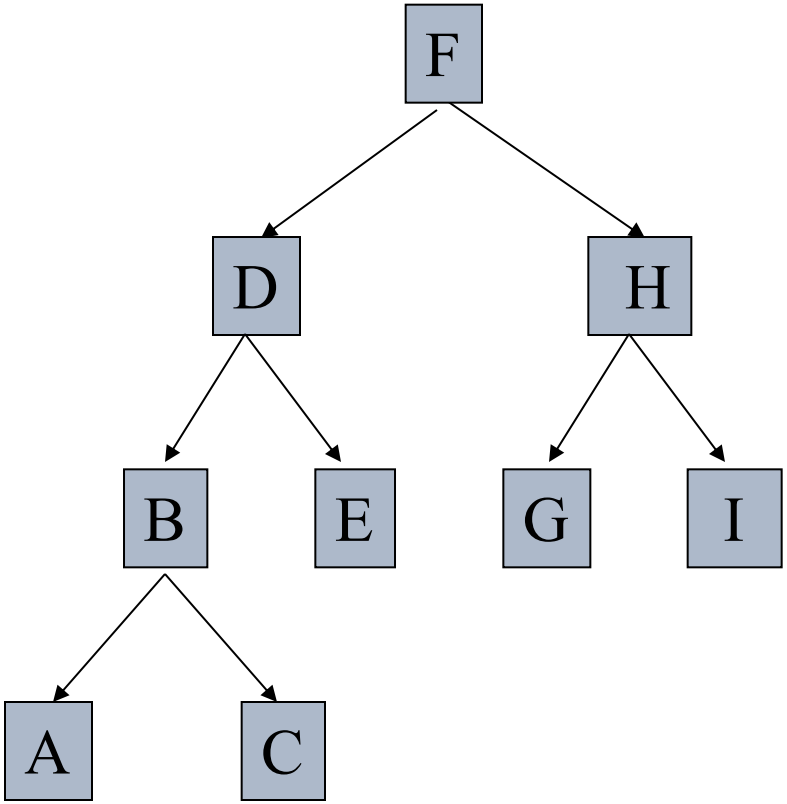
- A **complete binary tree** of depth d is a strictly binary tree with all of its leaves at level d.
- It is also called a **completely balanced** tree.
- A balanced tree of depth d contains 2^d leaves and 2^d -1 non-leaf nodes.



$2^2 = 4$
leaf nodes

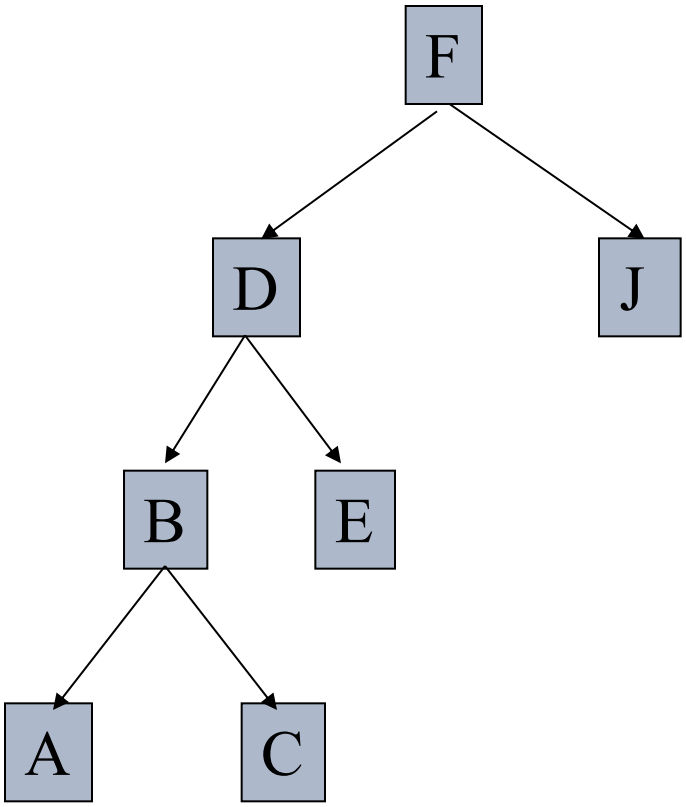
$2^2 - 1 = 3$
non-leaf nodes

Almost complete binary trees



- Each leaf is at level d or level d-1.
- If a node has a right descendant at level d, all the left descendants that are leaves are also at level d.

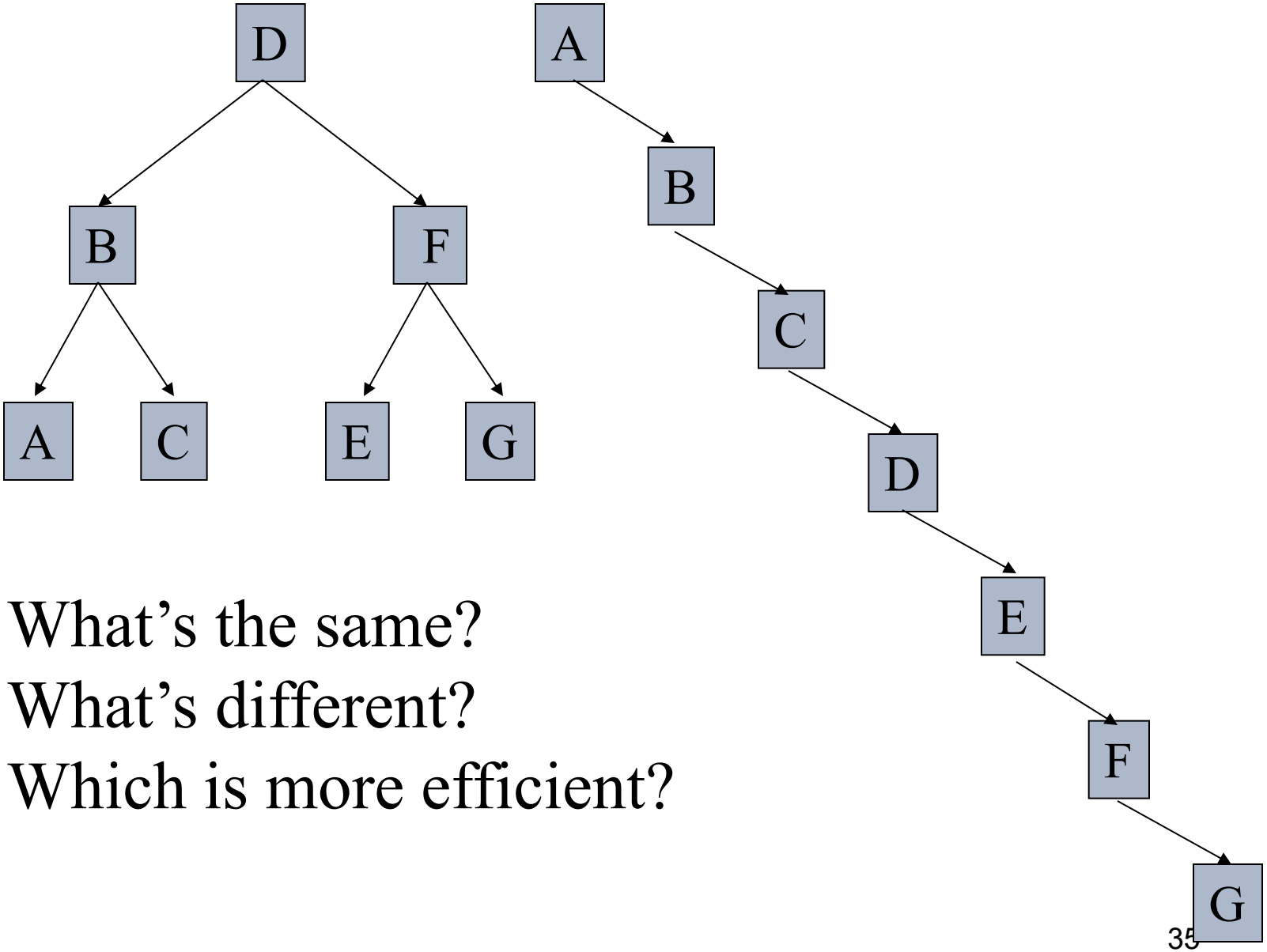
Is this tree “almost complete”?





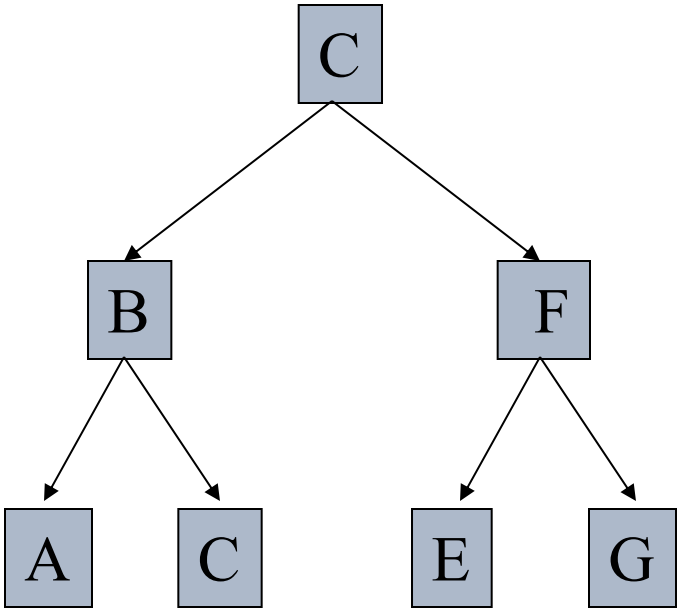
F has a right descendent at level d, but not all left leaf descendants are at level d!

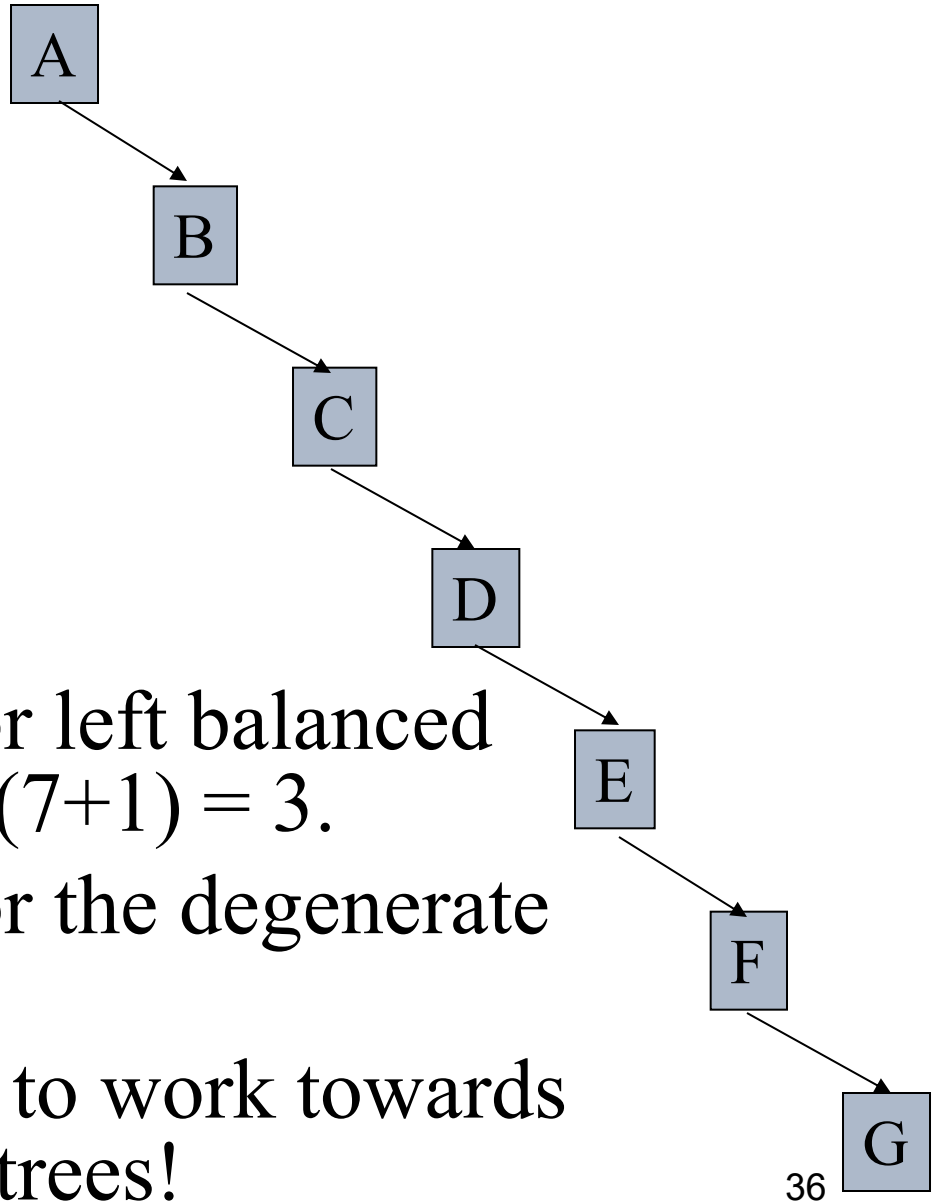
Two binary trees...



What's the same?
What's different?
Which is more efficient?

How good? How bad?





Worst case search for left balanced tree is depth $L=\log_2(7+1) = 3$.

Worst case search for the degenerate tree on right is L=7.

Insert & delete need to work towards producing balanced trees!

Producing balanced trees

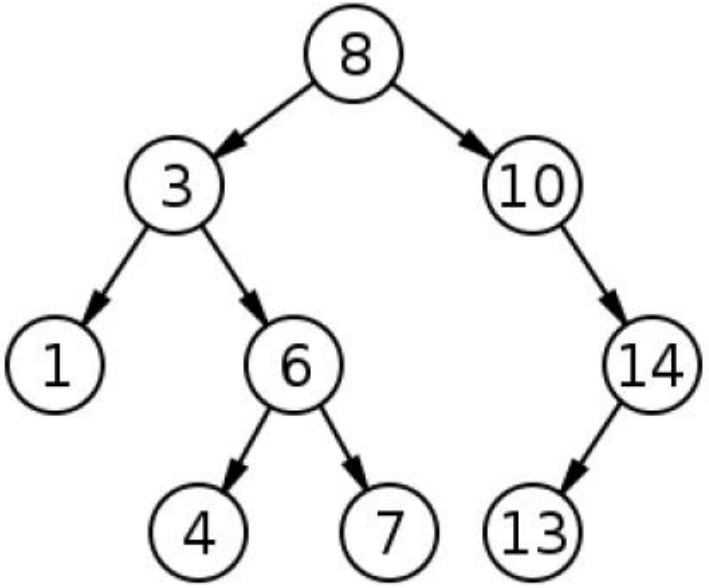
- Inserting generally produces roughly balanced trees if inserted into tree in random order.
- If the data is inserted in order, the tree degenerates into a linked list!
- Sometimes if the tree becomes unbalanced, it's appropriate to "rebuild" the tree.
- Insert to create balanced trees is complex, and will be covered in a later module.

Binary Search Trees

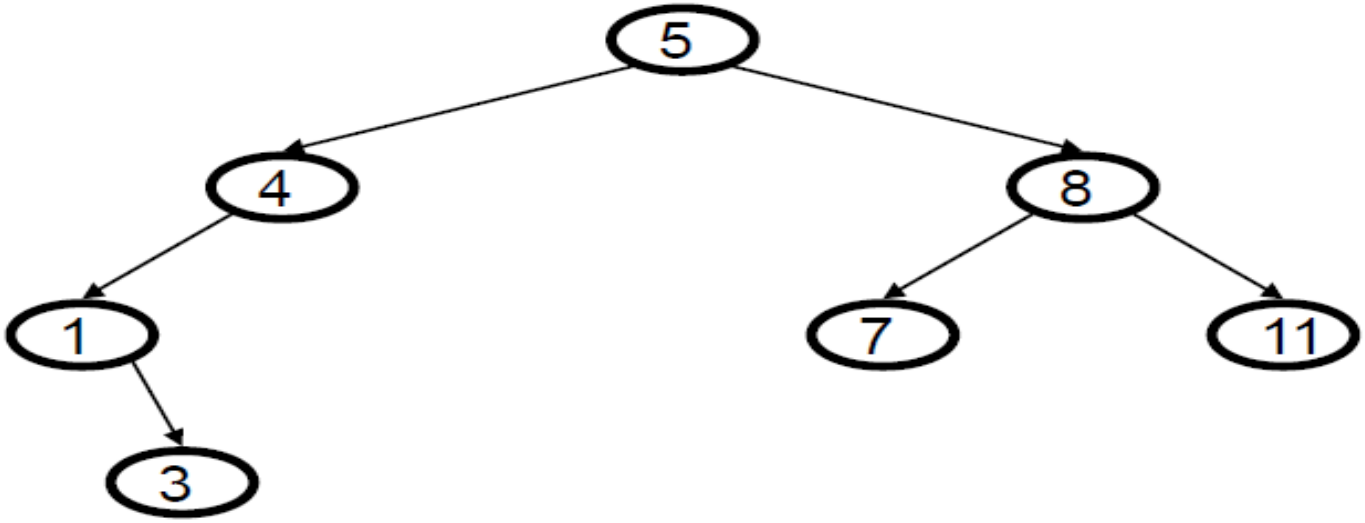
The binary search tree is a binary tree that satisfies the search order property as follows.

–The left sub tree of a node contains only nodes with keys less than the node's key.
–The right sub tree of a node contains only nodes with keys greater than the node's key.
–Both the left and right sub trees must also be binary search trees.

•Does not allow duplicates.

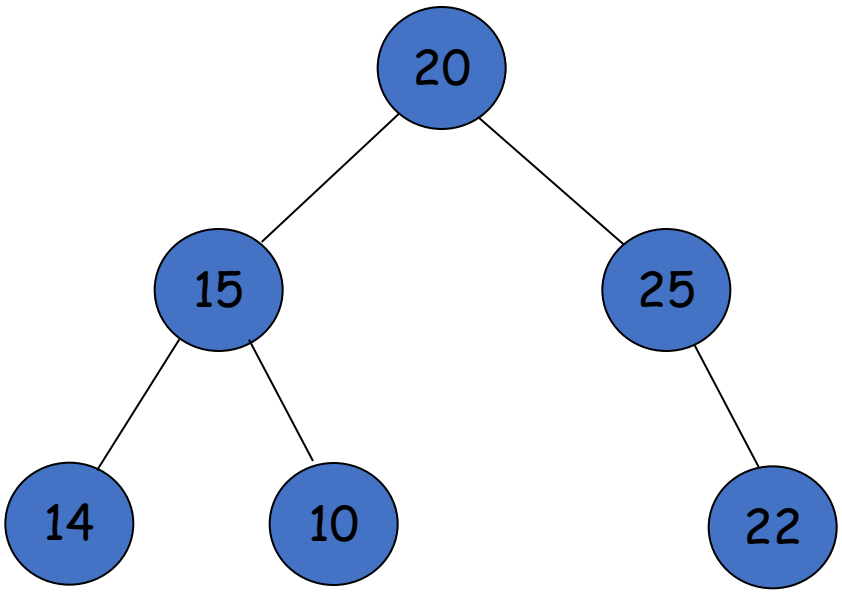


Example

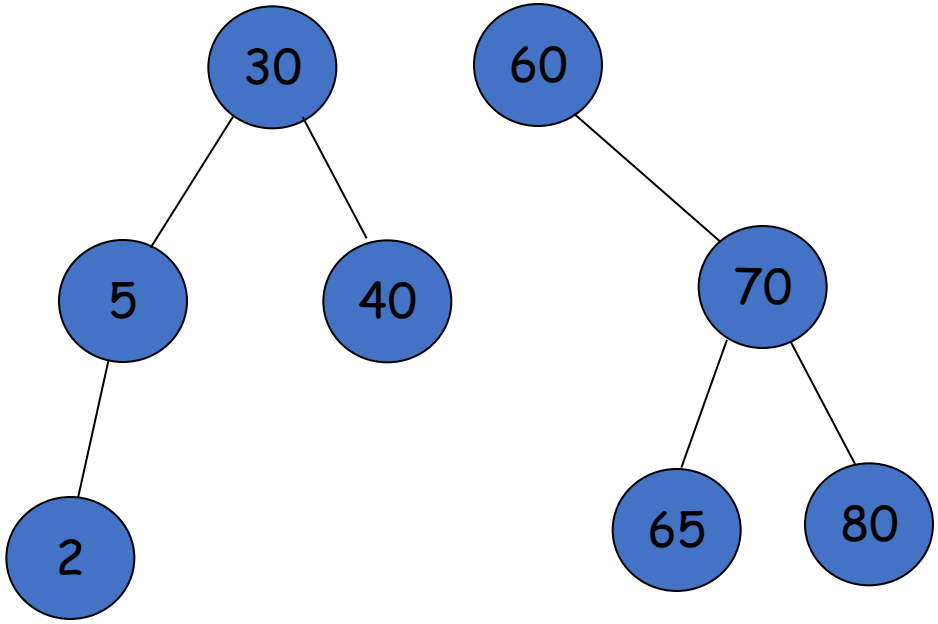


BINARY SEARCH TREE

Binary Search Trees

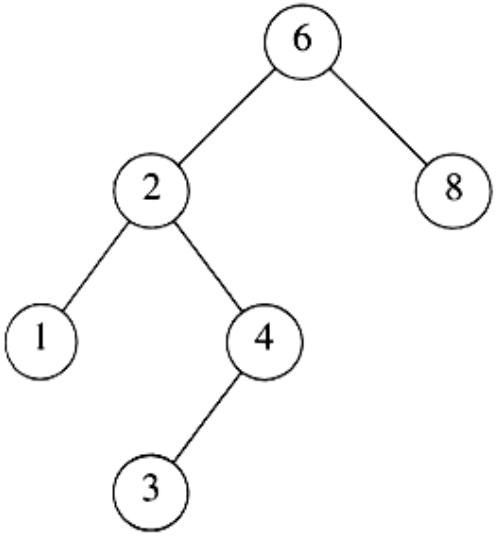


Not binary search tree

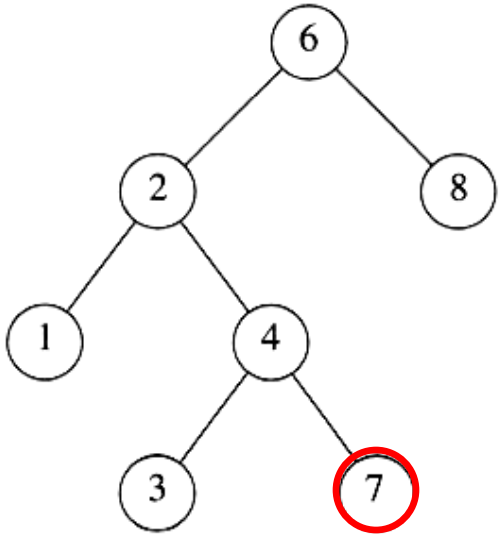


Binary search trees

Binary Search Trees



A binary search tree



Not a binary search tree

Binary Search Trees

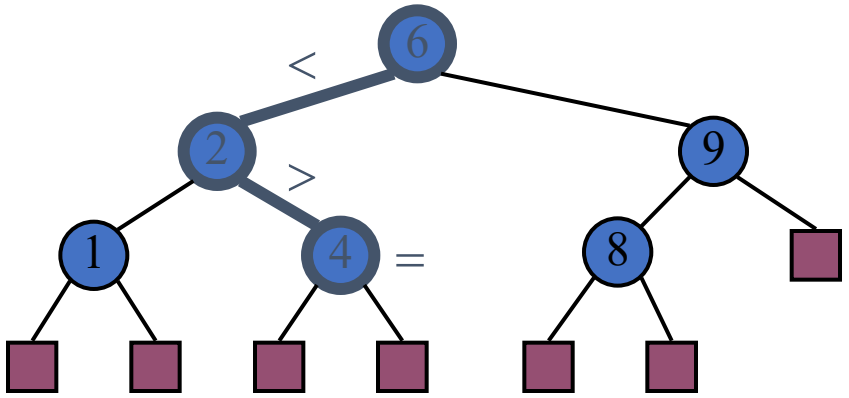
- Why using binary search tree
  - traverse in in-order: sorted list
  - searching becomes faster

- But..
  - Insert, delete: slow

Search

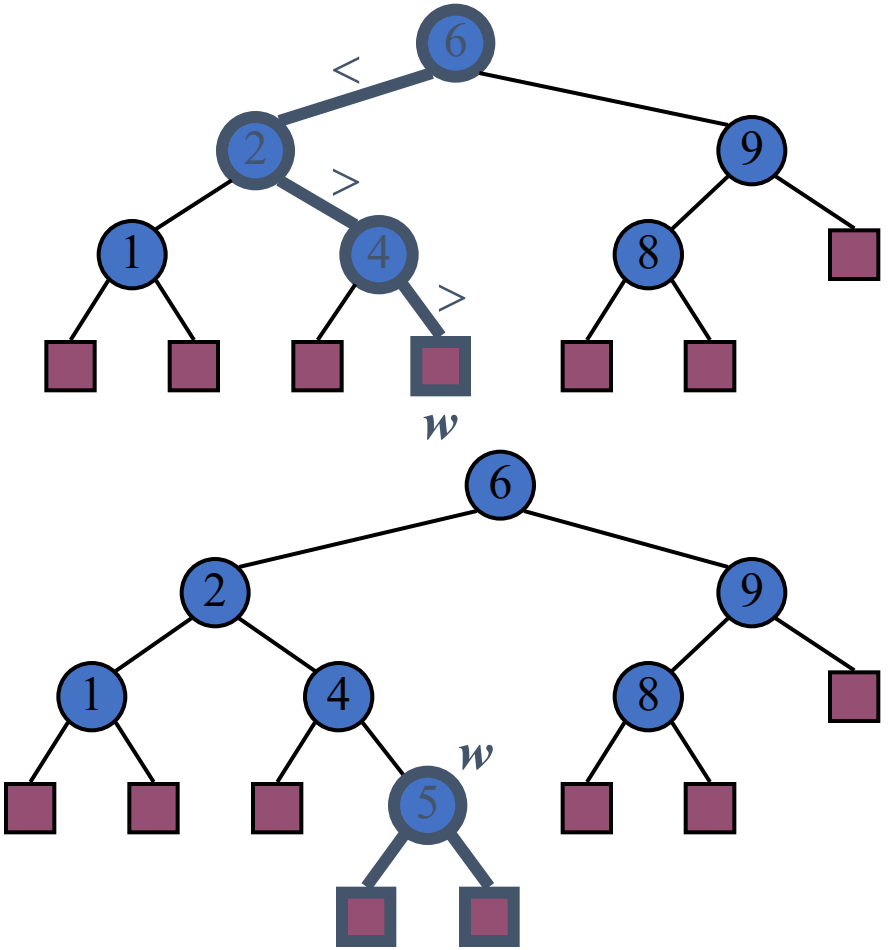
- To search for a key ***k***, we trace a downward path starting at the root
- The next node visited depends on the outcome of the comparison of ***k*** with the key of the current node
- If we reach a leaf, the key is not found and we return nukk
- Example: find(4):
  - Call TreeSearch(4,root)

```
Algorithm TreeSearch(k, v)
    if (v ==NULL)
        return v
    if k < key(v)
        return TreeSearch(k, T.left(v))
    else if k = key(v)
        return v
    else { k > key(v) }
        return TreeSearch(k, T.right(v))
```



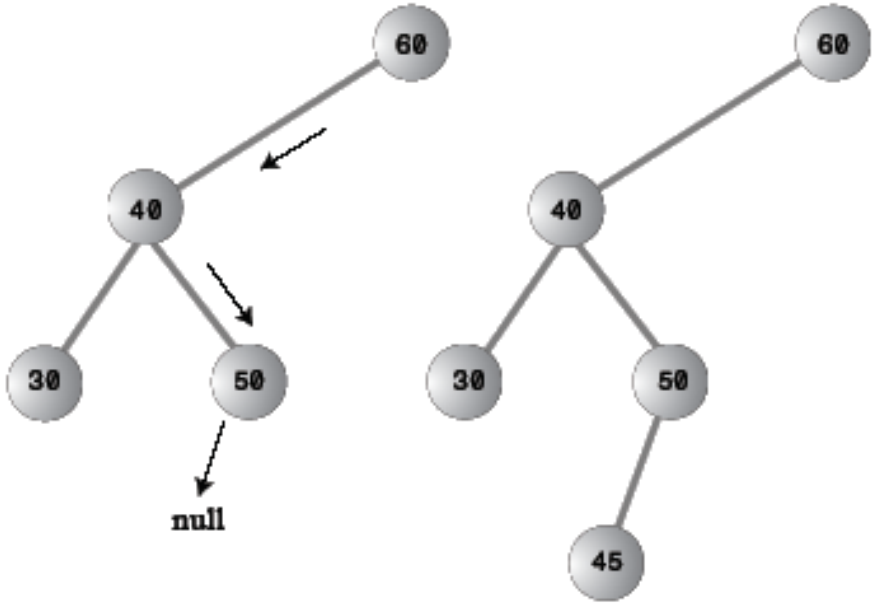
Insertion

- To perform operation insert(k, o), we search for key k (using TreeSearch)
- Assume k is not already in the tree, and let w be the leaf reached by the search
- We insert k at node w and expand w into an internal node
- Example: insert 5



Binary Search Trees

- Insert new node



Inserting a node.

a) Before insertion

b) After insertion

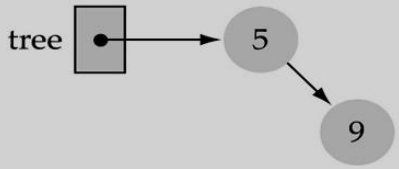
(a) tree



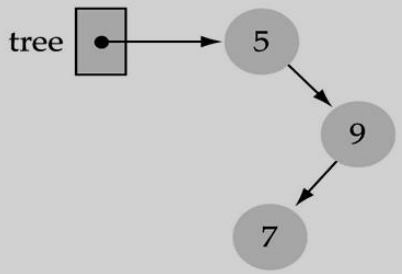
(b) Insert 5



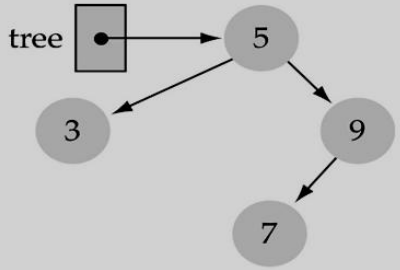
(c) Insert 9



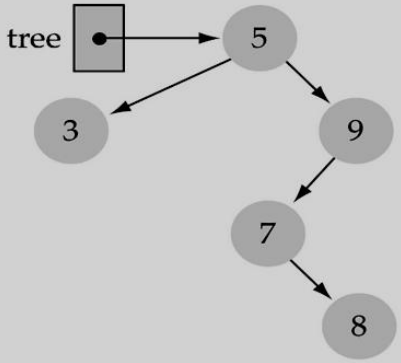
(c) Insert 7



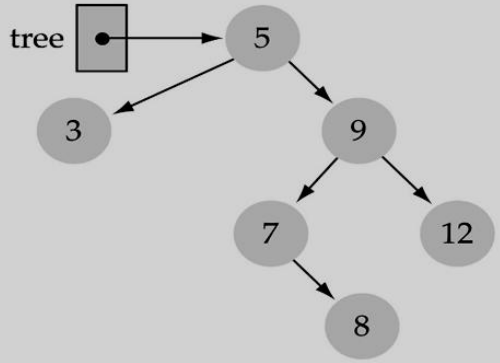
(e) Insert 3



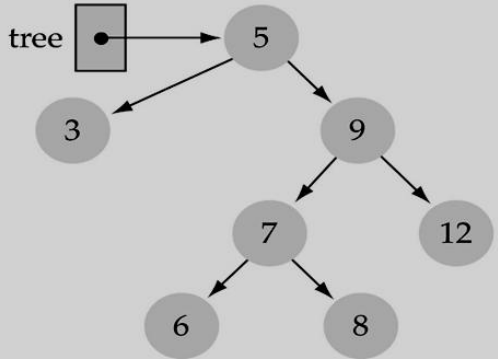
(f) Insert 8



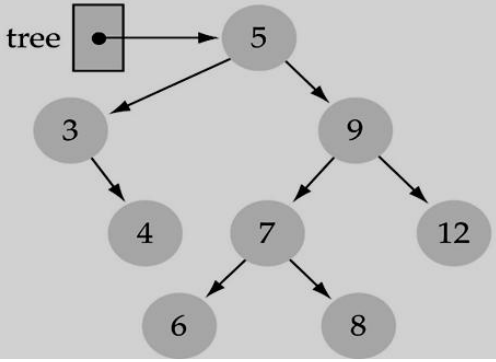
(g) Insert 12



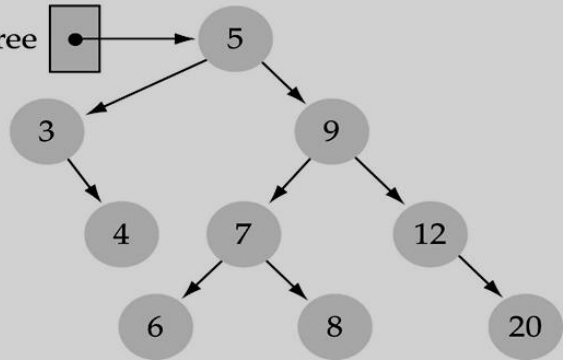
(h) Insert 6



(i) Insert 4



(j) Insert 20



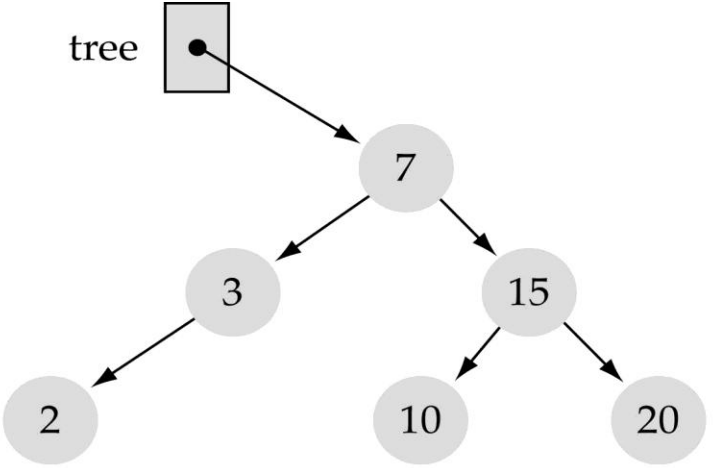
Binary Search Trees

- Insert new node

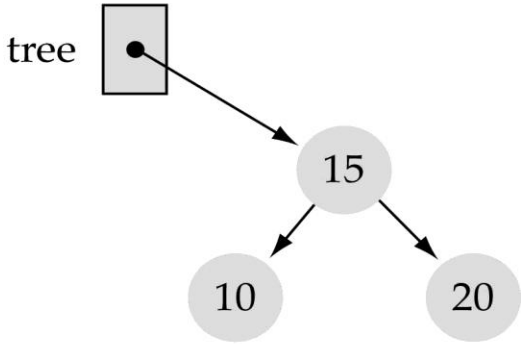
```
void InsertNode(node* &root,node *newnode)
{
  if (root==NULL)
        root=newnode;
  else
        if (root->data>newnode->data)
              InsertNode(root->l,newnode);
        else
              if (root->data<newnode->data)
                    InsertNode(root->r,newnode);
}
```

Insert node

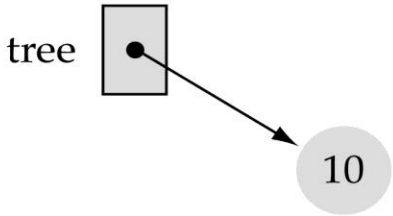
(a) The initial call



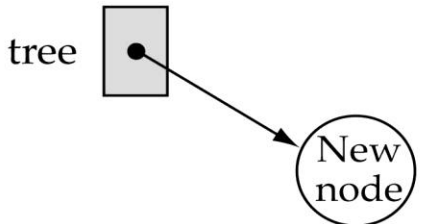
(b) The first recursive call



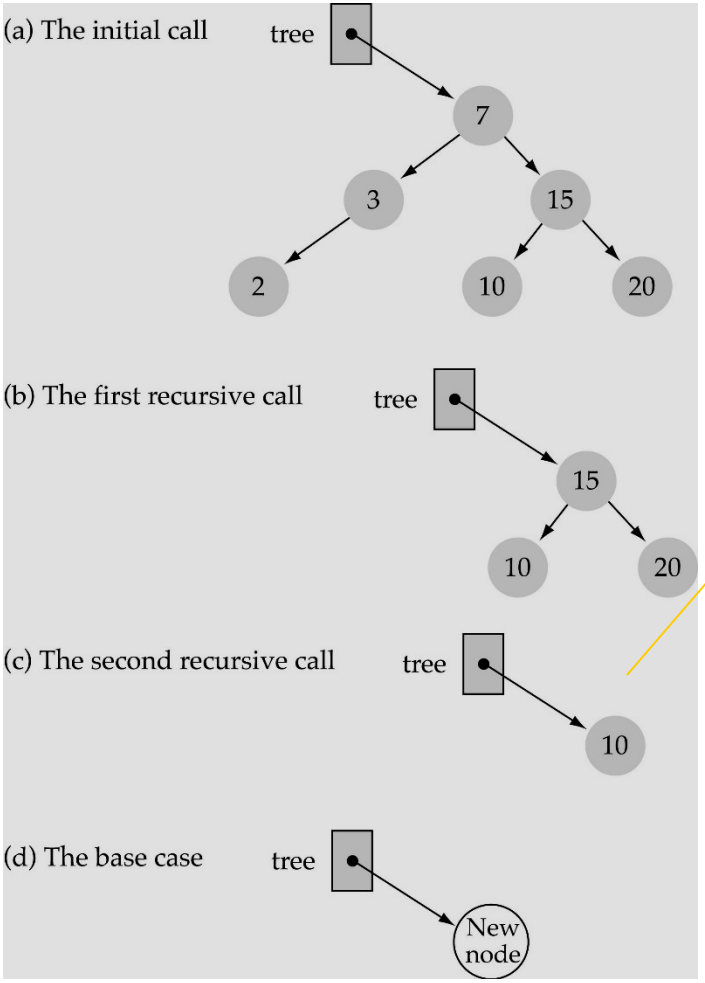
(c) The second recursive call

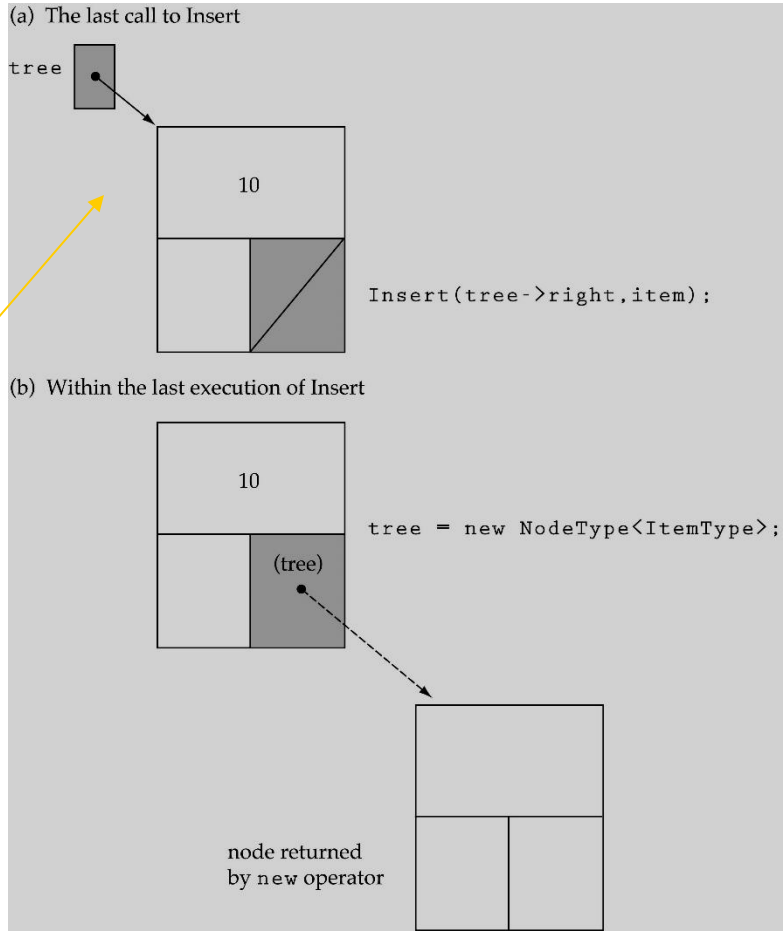


(d) The base case

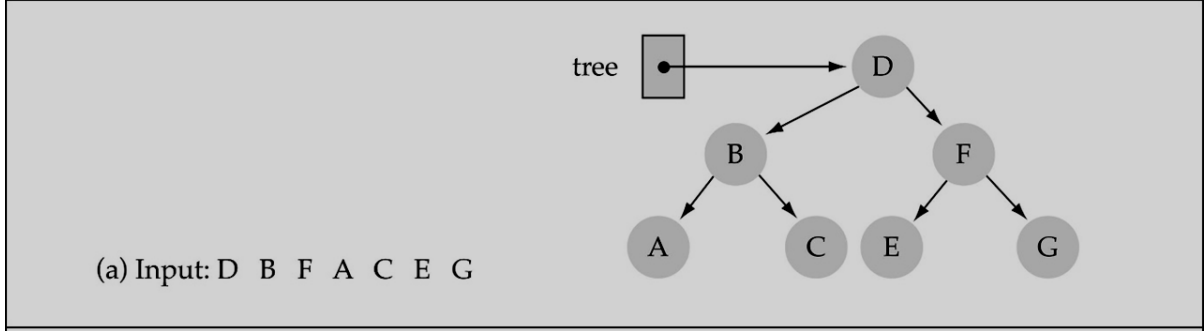


Insert node

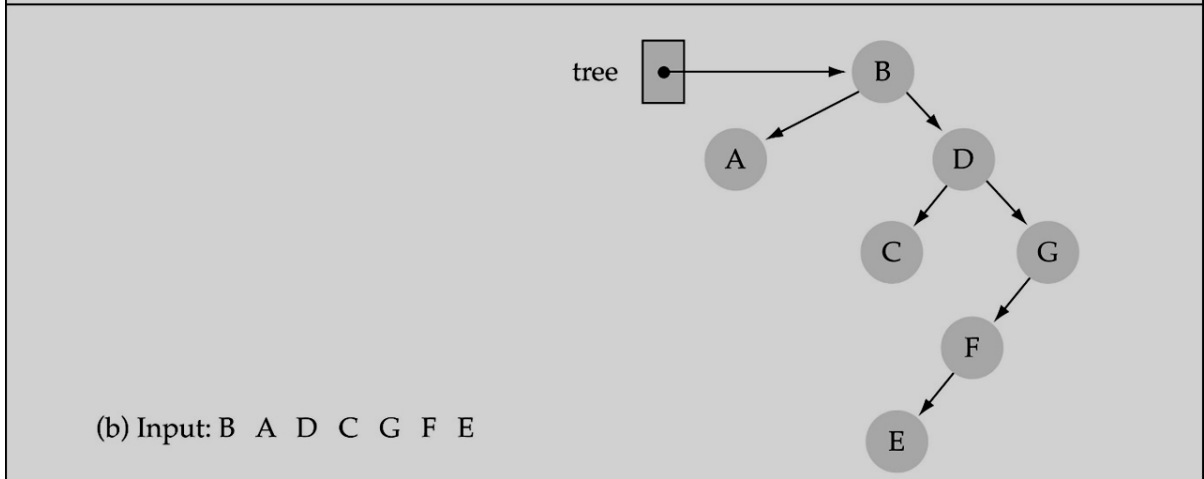

(a) The initial call
tree
7
3
15
2
10
20
(b) The first recursive call
tree
15
10
20
(c) The second recursive call
tree
10
(d) The base case
tree
New node


(a) The last call to Insert
tree
10
Insert(tree->right,item);
(b) Within the last execution of Insert
10
(tree)
tree = new NodeType<ItemType>;
node returned
by new operator

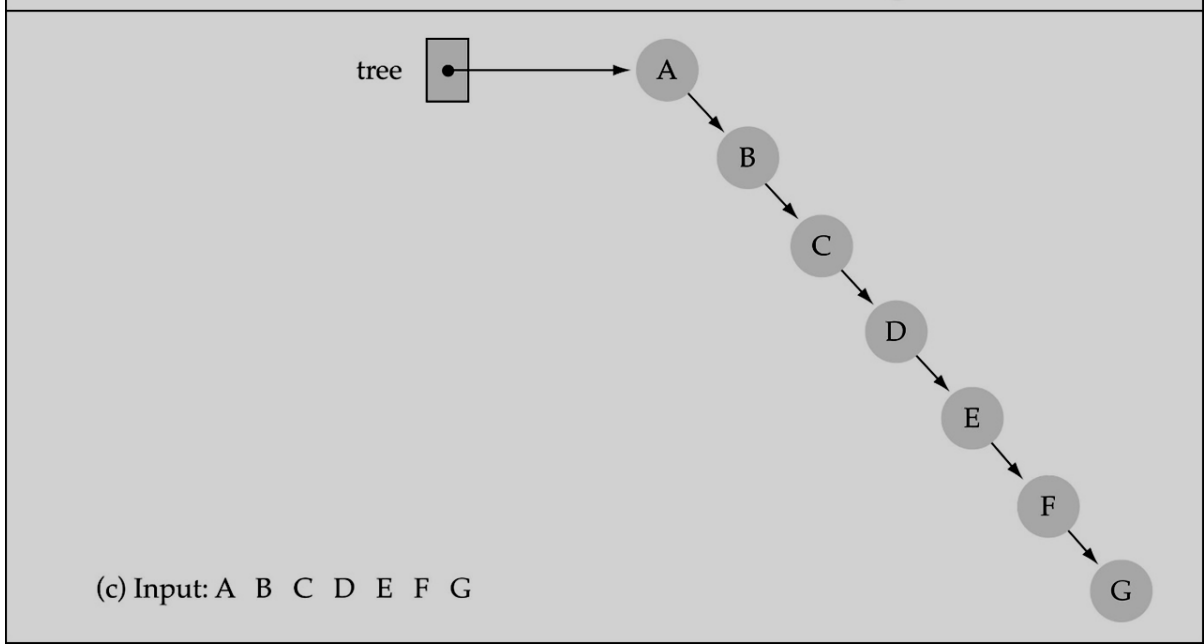
Insert Order



(a) Input: D B F A C E G



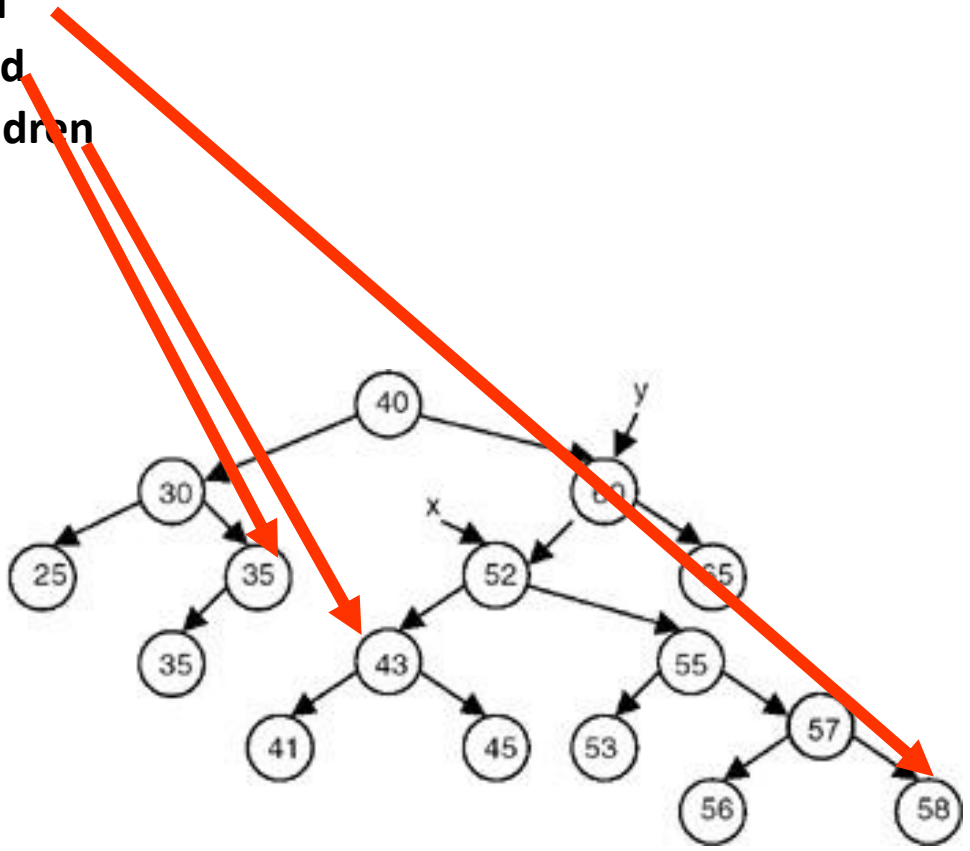
(b) Input: B A D C G F E



(c) Input: A B C D E F G

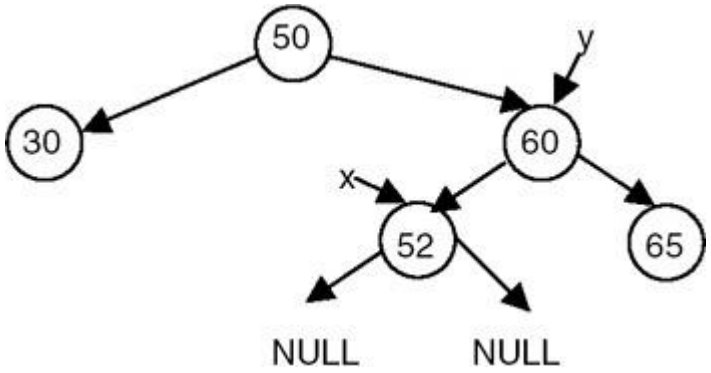
Delete node

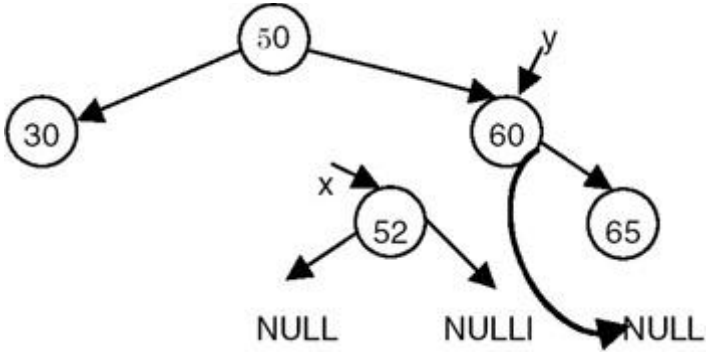
- **Divide into 3 cases**
  - **Deletion of a Node with No Child**
  - **Deletion of a Node with one Child**
  - **Deletion of a Node with two Children**



Delete node

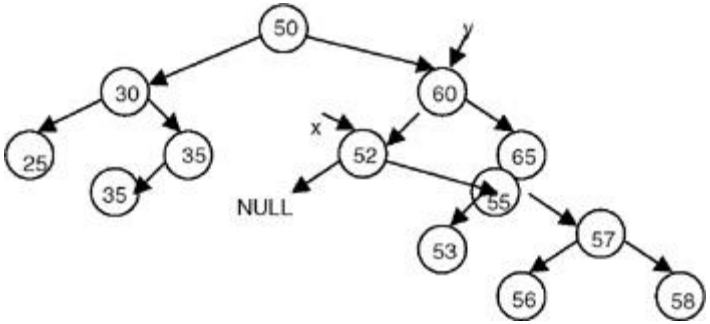
- **Deletion of a Node with No Child**
  - **Set the left of y to NULL**
  - **Dispose of the node pointed to by x**

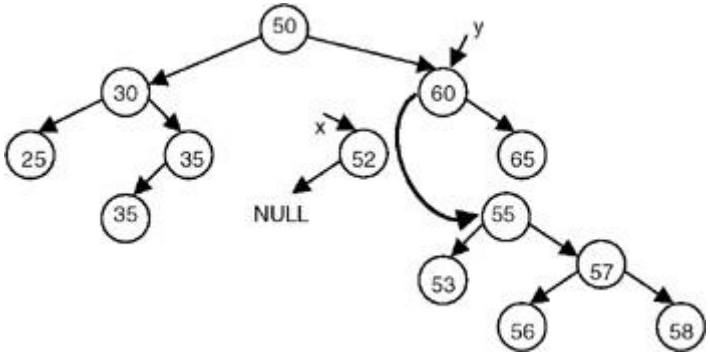




Delete node

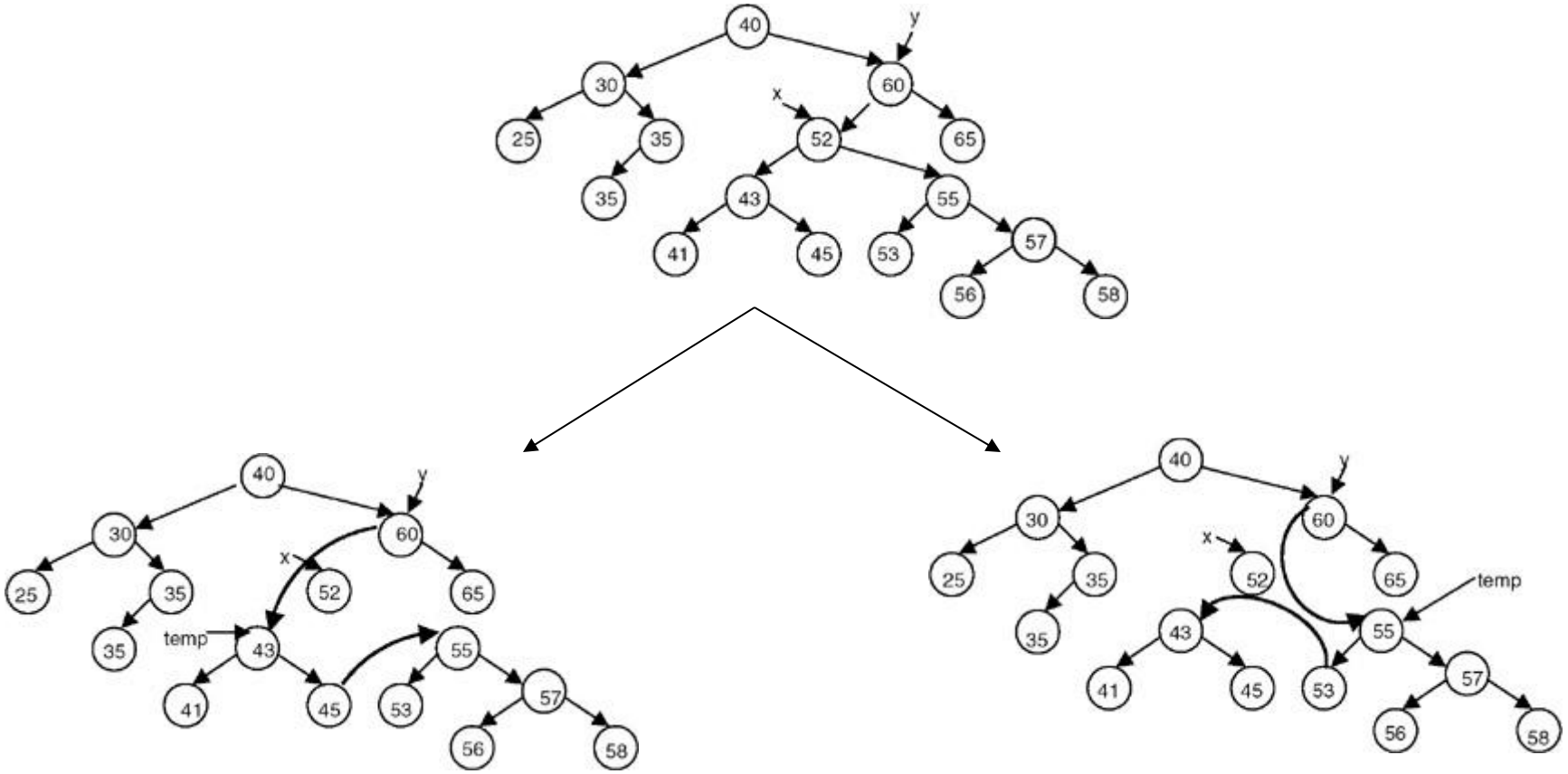
- **Deletion of a Node with One Child**
  - Make the y->left=x->right
  - dispose of the node pointed to by *x*





Delete node

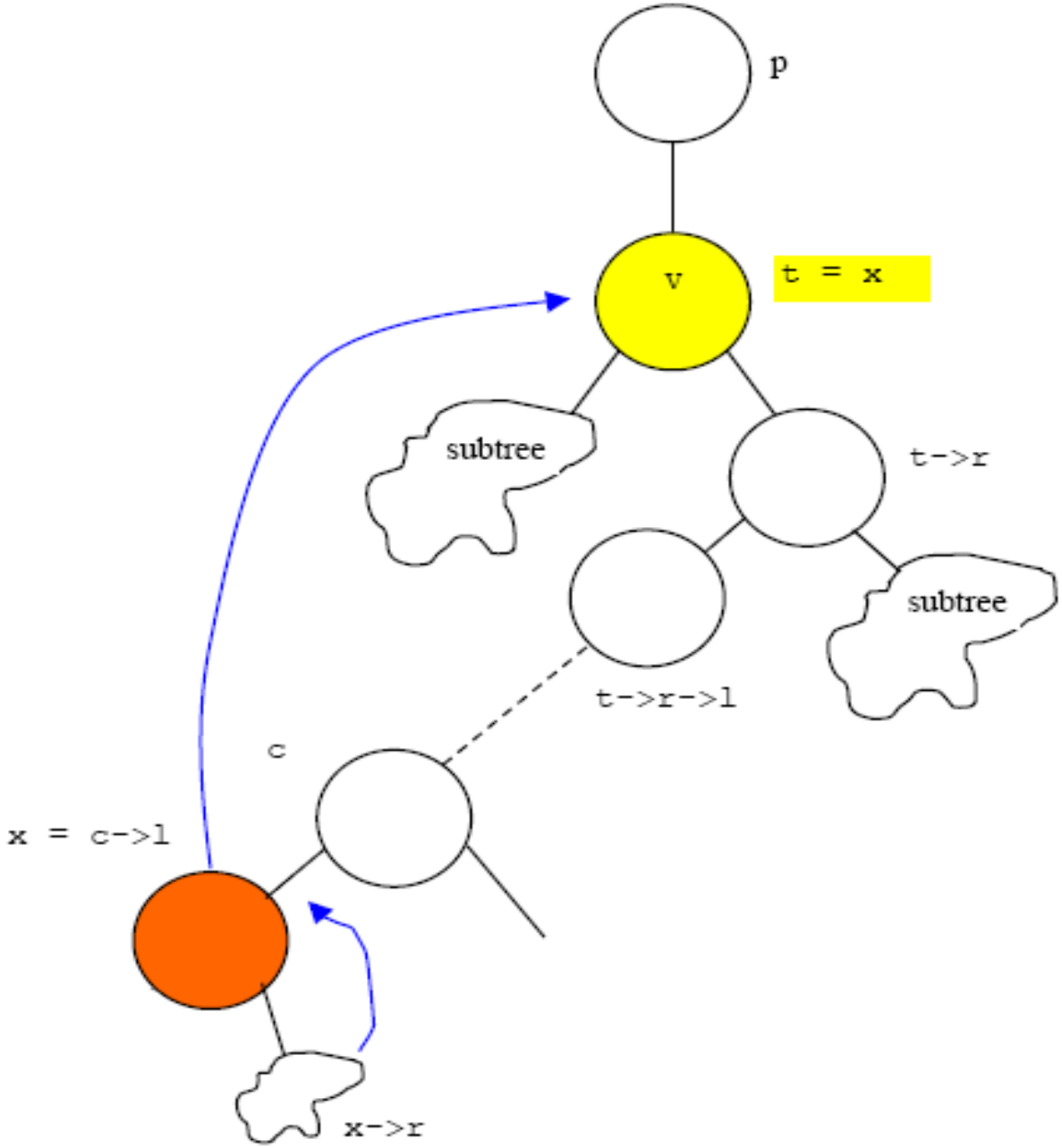
- **Deletion of a Node with two Children**



Delete node

rightmost child of the subtree of the left

leftmost child of the subtree of the right



Delete node

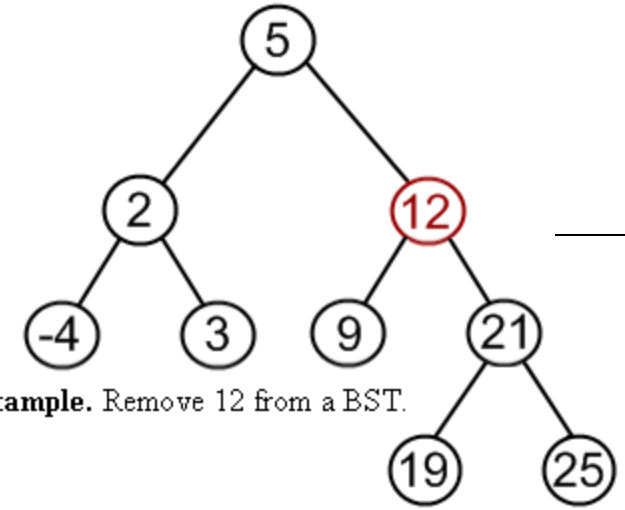
We can delete a node with two child nodes in two ways

In Order Predecessor

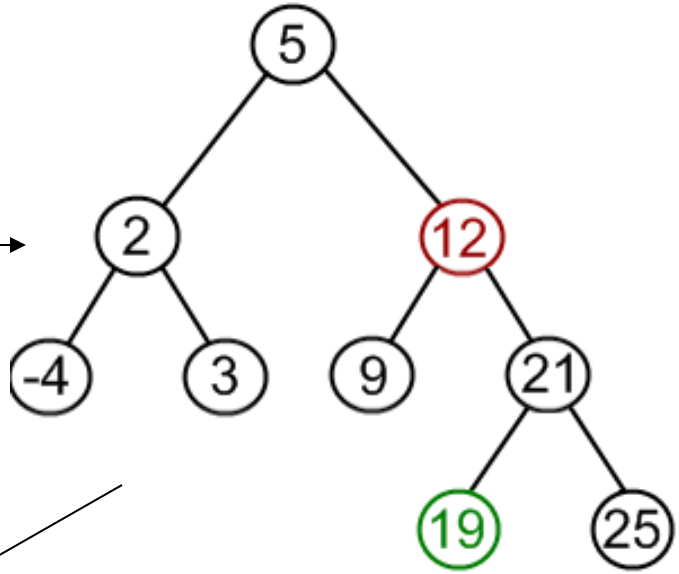
Largest element of the left sub tree

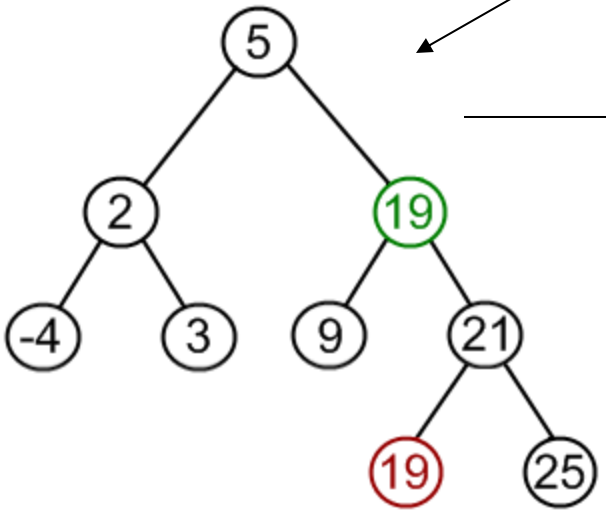
In Order Successor

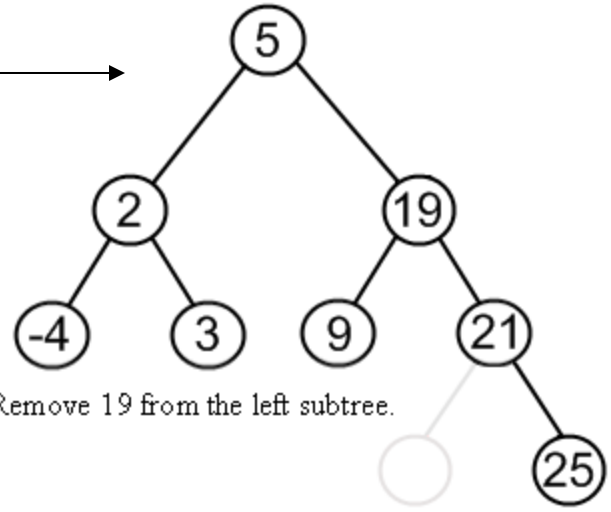
Smallest element of the right sub tree



Example. Remove 12 from a BST.

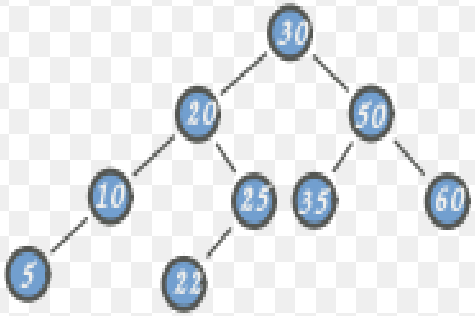




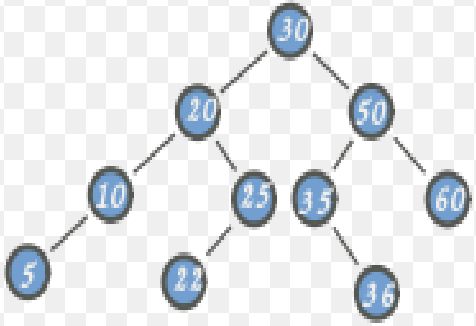


Remove 19 from the left subtree.

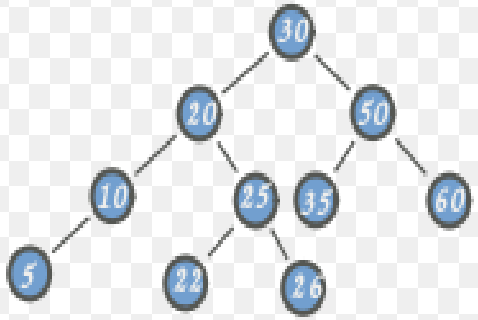
Base



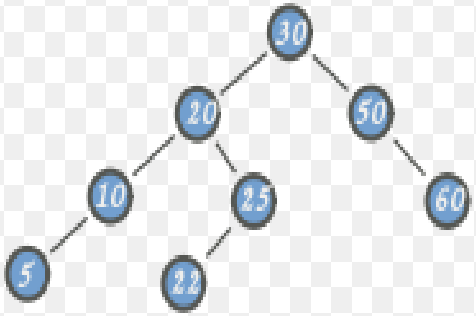
a) Insert 36



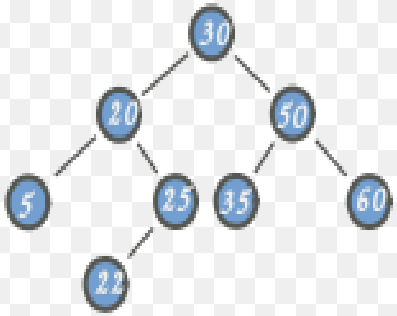
b) Insert 26



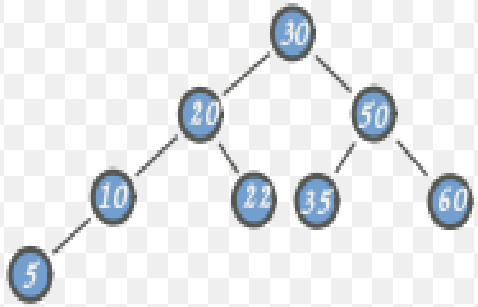
c) Delete 35



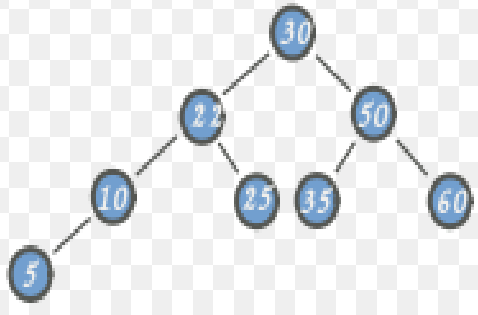
d) Delete 10



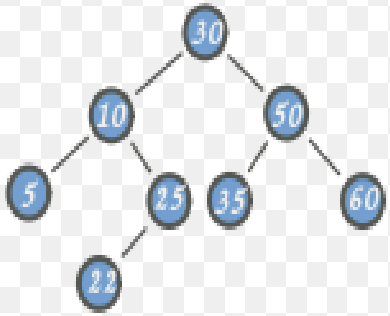
e) Delete 25



f) Delete 20 using in-order successor

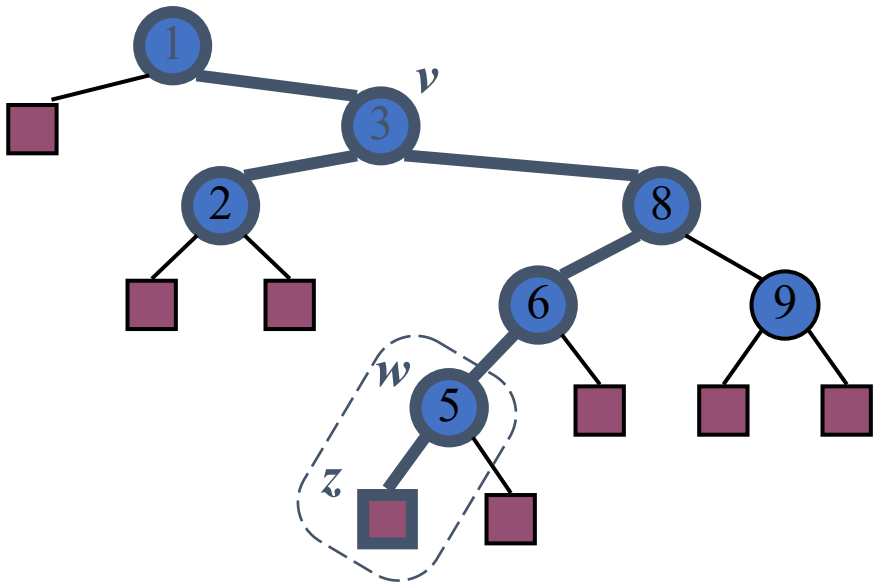


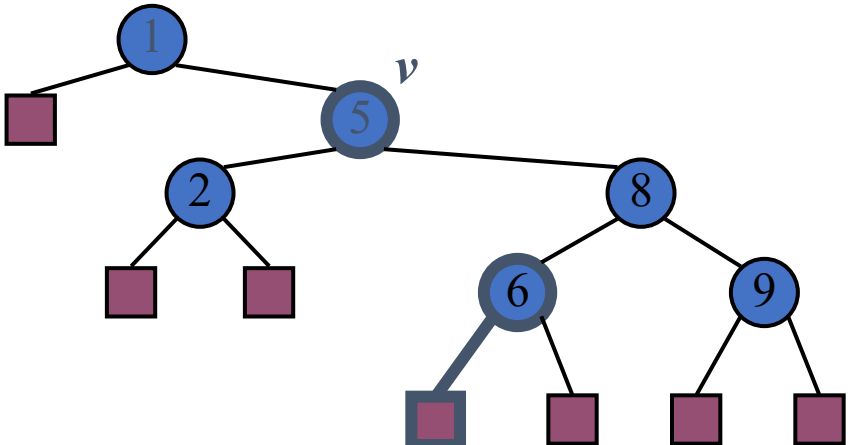
f) Delete 20 using in-order predecessor



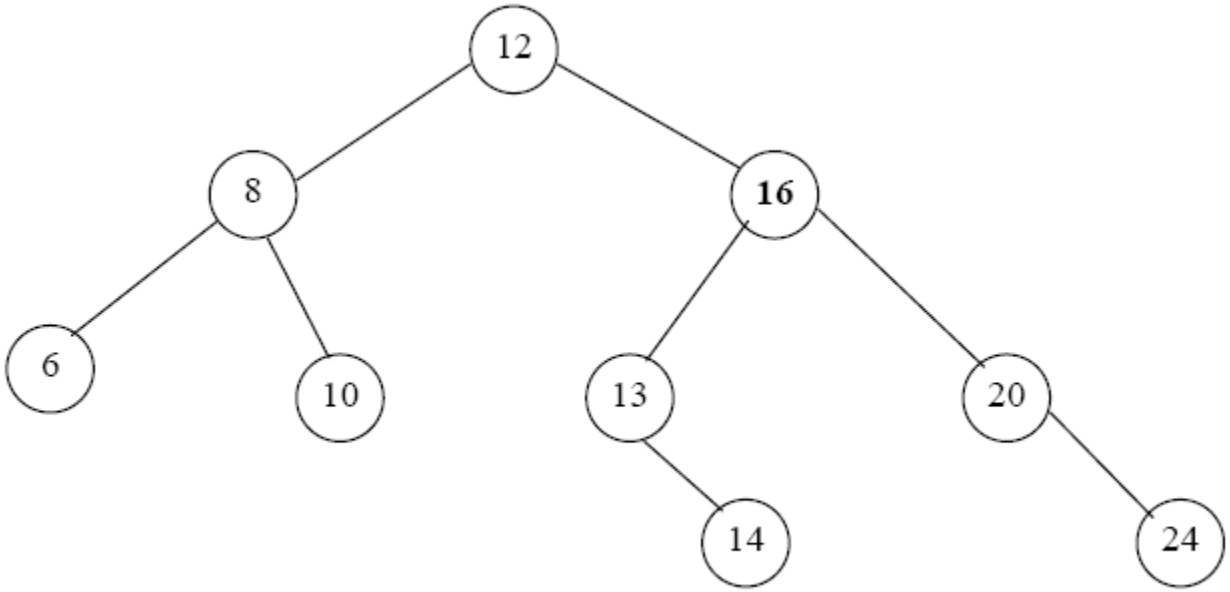
Deletion (cont.)

- We consider the case where the key k to be removed is stored at a node v whose children are both internal
  - we find the internal node w that follows v in an inorder traversal
  - we copy $key(w)$ into node v
  - we remove node w and its left child z (which must be a leaf) by means of operation removeExternal(z)
- Example: remove 3

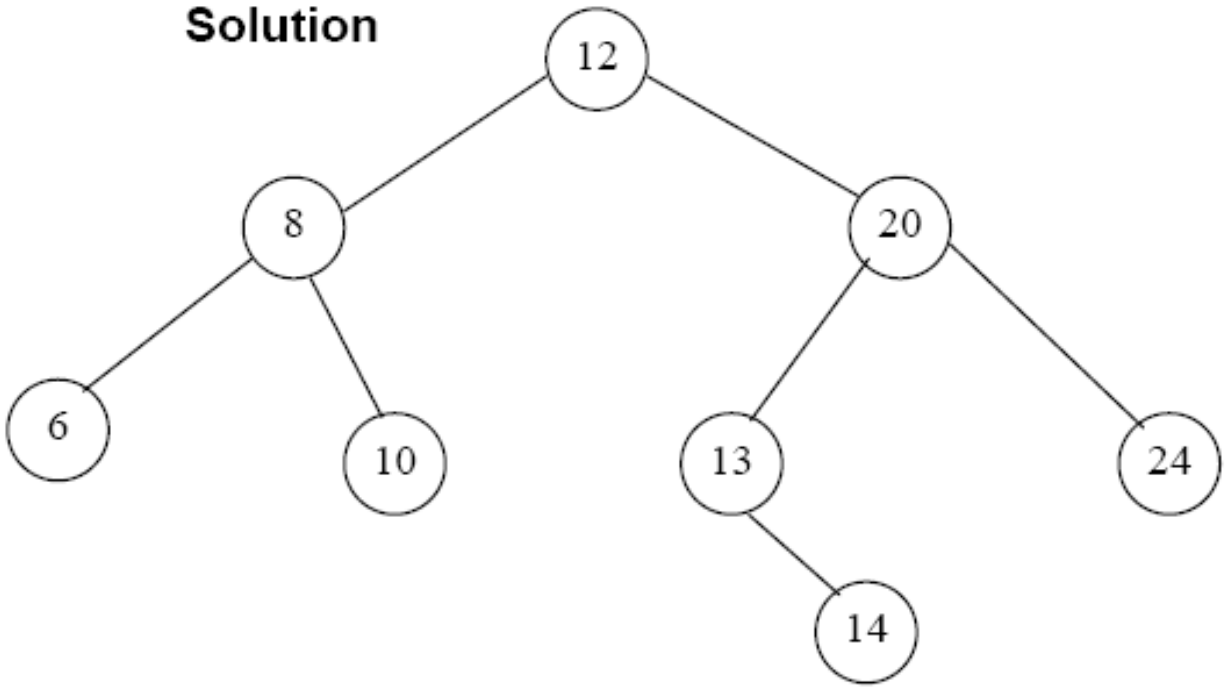




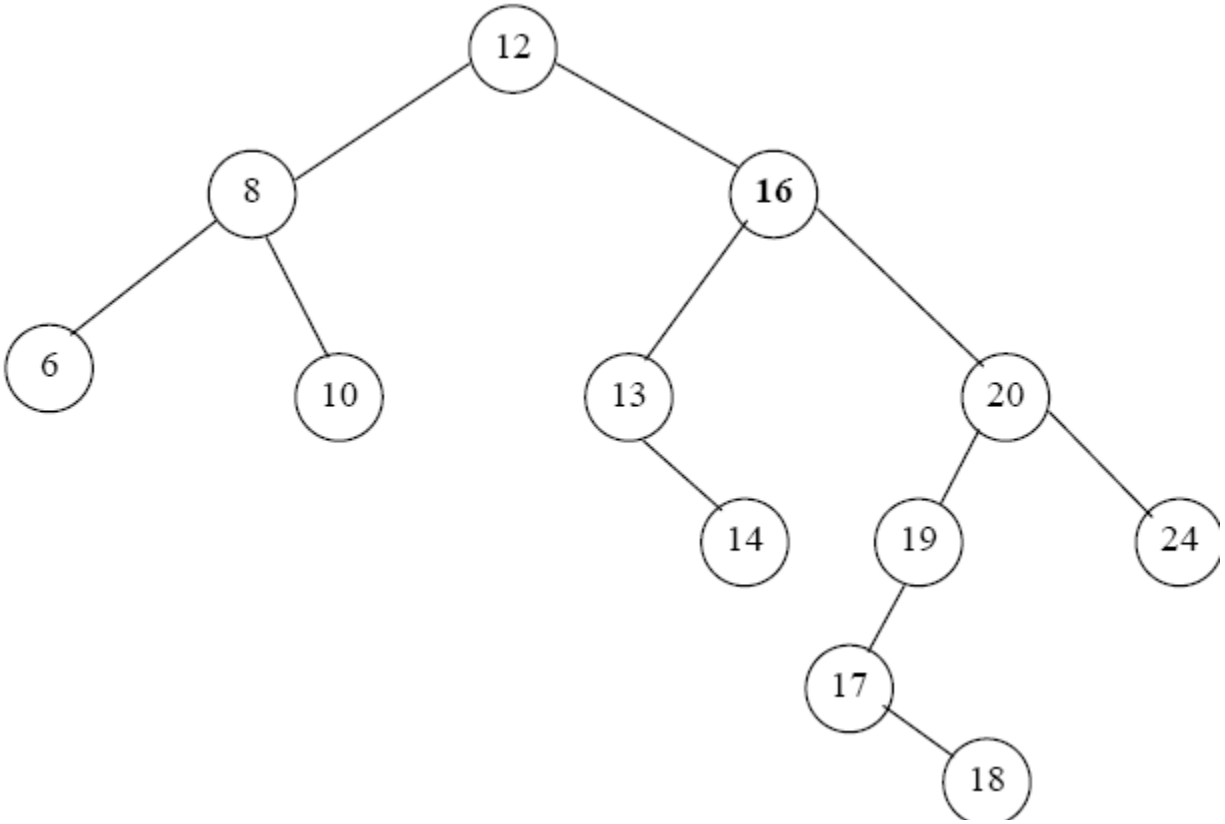
Deletion (cont.): exercise1: remove 16



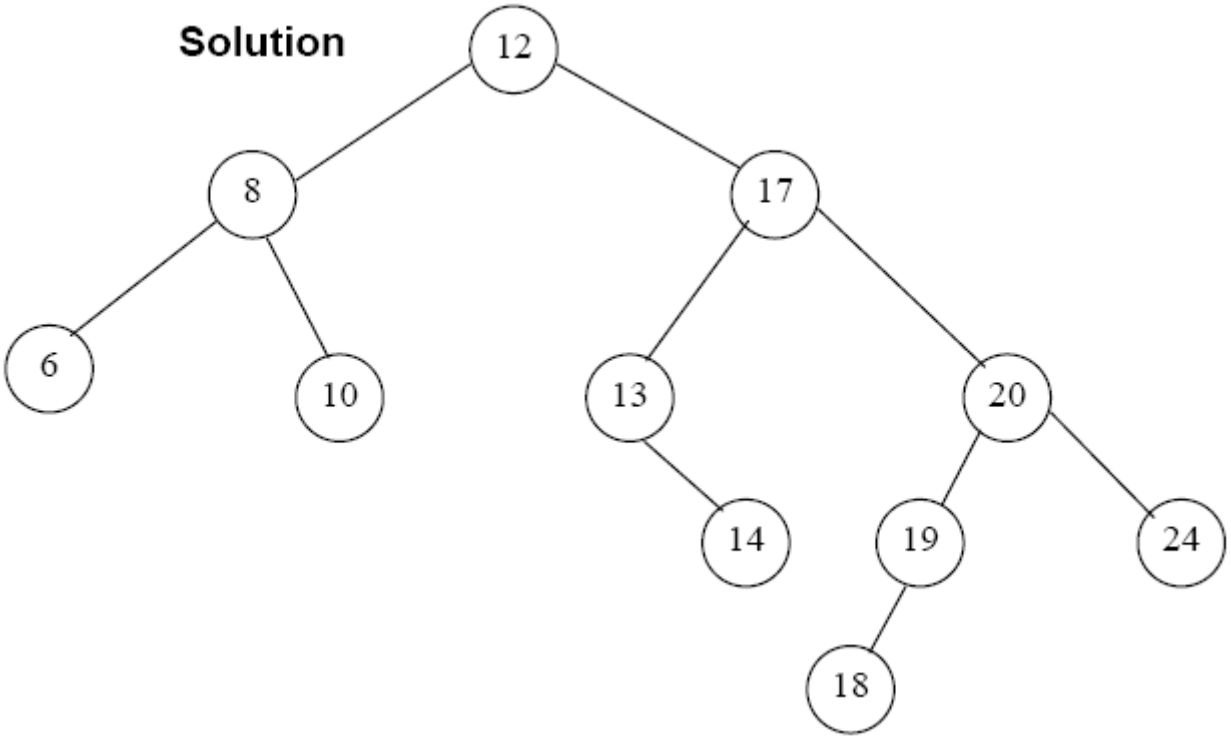
Deletion (cont.): exercise1: remove 16



Deletion (cont.): exercise2: remove 16



Deletion (cont.): exercise2: remove 16



END